# claude-windows / b55cb444-5f5e-4770-ad71-cee76683ff71

## Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\b55cb444-5f5e-4770-ad71-cee76683ff71.jsonl`
- SHA-256: `eb30fa62dd89b735b6d9ee81012b213b9d3436a81bfb44ee35397568800b84d3`
- Source modified: `2026-07-08T06:49:35+00:00`
- Imported at: `2026-07-08T15:59:19+00:00`
- project: `C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer`
- session_id: `b55cb444-5f5e-4770-ad71-cee76683ff71`

## Transcript

### 1. user (2026-07-08T06:16:28.778Z)

In backend/pipeline/stitcher/compiler.py, `_probe_encoder_available(encoder)` (added by #521, PR #525) runs a 1-frame test encode with `-i color=c=black:s=64x64:d=0.1` to decide whether an NVENC encoder (h264_nvenc/hevc_nvenc) is runnable, falling back to libx264 if not.

Bug: 64x64 is BELOW NVENC's minimum encode dimensions, so the probe FAILS ("Nothing was written into output file, because at least one of its streams received no packets. Conversion failed!") even on boxes where NVENC works fine at real resolutions. Empirically characterized on an RTX 2070 Super (ffmpeg 8.1.1): h264_nvenc probe at 64x64 FAIL, 128x128 FAIL, 145x49 PASS, 256x256 PASS, 320x240 PASS; 4K encode works. Same failures with a real testsrc2 source (not a `color`-source artifact).

Impact: on any GPU box whose NVENC minimum exceeds 64x64 ? potentially INCLUDING the L4 Cloud Run worker ? `_select_encoder` silently falls back to libx264, defeating #521's entire NVENC-on-L4 optimization (the 4K watermark re-encode stays on slow CPU libx264). This has likely never been caught because the 2026-07-08 CUJ that motivated #521 measured the control-plane (libx264) path, not a live L4 NVENC stitch.

Fix: bump the probe source to a safe resolution >= 256x256 (e.g. `color=c=black:s=256x256:d=0.1` or testsrc2 at 320x240) so the probe reliably reflects NVENC runnability. Keep it a real 1-frame encode to null (not `-encoders` grep). Add/extend a unit test in tests/test_compiler.py that asserts the probe command uses a >=256px resolution (guard against regressing back to a sub-minimum size). Verify on a real GPU box that h264_nvenc now probes PASS. Branch off origin/master; one PR referencing #521/#525.

### 2. assistant (2026-07-08T06:16:54.413Z)

I'll start by understanding the current repository state before making any changes. The git status snapshot shows unmerged paths (a merge conflict in progress) in the very file I need to edit, plus the current branch is unrelated to this task. Let me investigate carefully before touching anything.

### 3. user (2026-07-08T06:16:56.311Z)

```
On branch codex/issue-523-sd-reel-plan
Your branch is up to date with 'origin/master'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md
        modified:   SESSION_HANDOFF.md
        modified:   backend/config/models.py
        modified:   backend/infrastructure/database_jobs.py
        modified:   backend/main.py
        modified:   config.json
        modified:   tests/test_api_endpoints.py
```

```
        modified:   tests/test_async_jobs.py
        modified:   tests/test_compiler.py
        modified:   tests/test_watermark.py

Unmerged paths:
  (use "git restore --staged <file>..." to unstage)
  (use "git add <file>..." to mark resolution)
        both modified:   backend/orchestration.py
        both modified:   backend/pipeline/stitcher/compiler.py

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   backend/worker/__main__.py
        modified:   tests/test_compile_dispatch.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        score_golden_gate.py
        score_local.py
```

**4. user (2026-07-08T06:17:00.567Z)**

```
codex/issue-523-sd-reel-plan
---HEAD---
dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521) (#525)
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
(§9) (#522)
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload
(#518)
```

**5. user (2026-07-08T06:17:03.684Z)**

```
No merge in progress
---
stash@{0}: On codex/issue-523-sd-reel-plan: sd-reel-engine-wip
```

**6. assistant (2026-07-08T06:18:03.781Z)**

This is an important state to understand before I touch anything. Key observations:

1. **No merge is actually in progress** (no `MERGE_HEAD`), yet there are unmerged paths (`UU`) in `orchestration.py` and `compiler.py` ? the exact file I need to edit. Combined with the `stash@{0}: sd-reel-engine-wip` and the branch name `codex/issue-523-sd-reel-plan`, this is clearly a **sibling session's / owner's WIP for a different task** (SD reel engine, issue #523), left in a conflicted state.
2. My task (NVENC probe fix) is unrelated to that WIP, and the conflict sits right in my target file.

Per my safety conventions I must **not clobber this WIP** and must **branch cleanly off `origin/master`**. The clean way to do that without disturbing the dirty tree is a **separate git worktree**. Let me first fetch and inspect the real base state.

**7. user (2026-07-08T06:18:11.947Z)**

```
===origin/master log===
dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521) (#525)
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
(§9) (#522)
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload
(#518)
71c88ef docs(runlogs): validation-latency closure report for #512 + #513 (cloud-serving) (#519)
```

```
115b83d docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515) (#517)
```

**8. user (2026-07-08T06:18:16.179Z)**

```
wrote compiler_master.py
626 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-khelsutra-guru-badminton-
highlight-indexer/b55cb444-5f5e-4770-ad71-cee76683ff71/scratchpad/compiler_master.py
```

**9. user (2026-07-08T06:18:19.908Z)**

```
wrote test_compiler_master.py
224 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-khelsutra-guru-badminton-
highlight-indexer/b55cb444-5f5e-4770-ad71-cee76683ff71/scratchpad/test_compiler_master.py
```

**10. assistant (2026-07-08T06:18:25.482Z)**

```
origin/master is at `dac6736` (same commit; all the mess is uncommitted WIP). Now let me examine
the actual buggy code on the clean base.
```

**11. user (2026-07-08T06:18:28.338Z)**

```
31:def _probe_encoder_available(encoder: str) -> bool:
51:                "color=c=black:s=64x64:d=0.1",
86:        # works everywhere; "h264_nvenc"/"hevc_nvenc" use the L4's HARDWARE encoder for the
4K
88:        # (_select_encoder) and falls back to libx264 if this ffmpeg build can't run it.
277:    def _select_encoder(self) -> Tuple[List[str], str]:
284:        if enc in ("h264_nvenc", "hevc_nvenc") and not _probe_encoder_available(enc):
526:            video_codec_args, chosen_encoder = self._select_encoder()
```

**12. user (2026-07-08T06:18:29.920Z)**

```
2:where the f-string SyntaxError lived undetected). ffmpeg/ffprobe are mocked."""
48:        if cmd[0] == "ffprobe":
74:        # Create whatever output file ffmpeg/ffprobe was told to write so existence checks
pass.
75:        if cmd[0] == "ffprobe":
90:    # Per-clip trims + one concat = at least 3 ffmpeg invocations (plus an ffprobe).
100:        if cmd[0] == "ffprobe":
129:        if cmd[0] == "ffprobe":
166:def test_select_encoder_default_is_libx264(tmp_path):
167:    """Default encoder = libx264 (CPU) with the historical settings ? no probe, works
everywhere."""
169:    args, chosen = comp._select_encoder()
174:def test_select_encoder_nvenc_when_available(tmp_path, monkeypatch):
175:    """h264_nvenc resolves to the hardware encoder args when the probe reports it runnable
(the L4)."""
178:    monkeypatch.setattr(comp_mod, "_probe_encoder_available", lambda enc: True)
179:    comp = VideoCompiler(str(tmp_path / "out"), encoder="h264_nvenc")
180:    args, chosen = comp._select_encoder()
181:    assert chosen == "h264_nvenc"
182:    assert args[:2] == ["-c:v", "h264_nvenc"]
186:def test_select_encoder_falls_back_to_libx264_when_nvenc_unavailable(tmp_path, monkeypatch):
191:    monkeypatch.setattr(comp_mod, "_probe_encoder_available", lambda enc: False)
192:    comp = VideoCompiler(str(tmp_path / "out"), encoder="h264_nvenc")
193:    args, chosen = comp._select_encoder()
200:    carries -c:v h264_nvenc, and the concat stays -c copy."""
203:    monkeypatch.setattr(comp_mod, "_probe_encoder_available", lambda enc: True)
204:    comp = VideoCompiler(str(tmp_path / "out"), encoder="h264_nvenc")
210:        if cmd[0] == "ffprobe":
223:        assert "h264_nvenc" in cmd  # hardware encode on every clip
```

**13. assistant (2026-07-08T06:18:34.425Z)**

Let me read the full probe function and the relevant test in detail.

**14. user (2026-07-08T06:18:37.497Z)**

```
25
26      #: Per-encoder availability memo (#521): probing ffmpeg is a subprocess, so a run's
first clip pays
27      #: it and the rest reuse the verdict. Keyed by encoder name; cleared only across
processes.
28      _ENCODER_AVAILABLE: Dict[str, bool] = {}
29
30
31      def _probe_encoder_available(encoder: str) -> bool:
32          """Can this ffmpeg build actually RUN ``encoder`` (not merely list it)? Runs a tiny
1-frame
33          lavfi test encode to null. NVENC is often *compiled in* on boxes without an NVIDIA
GPU, so a
34          real test encode ? not ``-encoders`` grep ? is what distinguishes "usable" from
"will fail at
35          runtime". Memoized. Any error / timeout ? unavailable (caller falls back to
libx264)."""
36          if encoder in _ENCODER_AVAILABLE:
37              return _ENCODER_AVAILABLE[encoder]
38          ok = False
39          # The probe is a tiny 1-frame encode ? bound it to a short timeout (? the configured
ffmpeg
40          # budget, or 30 s when unbounded) so a wedged encoder can't hang the whole stitch.
41          configured = ffmpeg_timeout_sec()
42          probe_timeout = min(30.0, float(configured)) if configured else 30.0
43          try:
44              proc = subprocess.run(
45                  [
46                      ffmpeg_bin(),
47                      "-hide_banner",
48                      "-f",
49                      "lavfi",
50                      "-i",
51                      "color=c=black:s=64x64:d=0.1",
52                      "-frames:v",
53                      "1",
54                      "-c:v",
55                      encoder,
56                      "-f",
57                      "null",
58                      "-",
59                  ],
60                  capture_output=True,
61                  timeout=probe_timeout,
62              )
63              ok = proc.returncode == 0
64          except Exception:  # noqa: BLE001 ? a probe failure/timeout just means "not usable ?
fall back"
65              ok = False
66          _ENCODER_AVAILABLE[encoder] = ok
67          return ok
68
69
70      class VideoCompiler:
71          def __init__(
72              self,
73              output_dir: str,
74              end_padding: float = 1.0,
75              begin_padding: float = 0.5,
76              watermark: Optional[Any] = None,
77              encoder: str = "libx264",
```

```
78          ):
79              self.output_dir = output_dir
80              self.end_padding = end_padding
81              self.begin_padding = begin_padding
82              # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
83              # normalized to a dict (or None) so this module stays config-package-agnostic.
84              self.watermark = self._normalize_watermark(watermark)
85              # Video encoder for the per-clip trim re-encode (#521). "libx264" (CPU) is the
default and
86              # works everywhere; "h264_nvenc"/"hevc_nvenc" use the L4's HARDWARE encoder for
the 4K
87              # re-encode the watermark forces. The requested encoder is PROBED once per run
88              # (_select_encoder) and falls back to libx264 if this ffmpeg build can't run it.
89              self.encoder = (encoder or "libx264").strip() or "libx264"
90              os.makedirs(output_dir, exist_ok=True)
91
92          @staticmethod
93          def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
94              if watermark is None:
95                  return None
96              if hasattr(watermark, "model_dump"):
97                  return watermark.model_dump()
98              if isinstance(watermark, dict):
99                  return dict(watermark)
100             return None
101
102         @staticmethod
103         def _ff_quote(path: str) -> str:
104             """Make a filesystem path safe to embed inside single quotes in an ffmpeg
```

**15. user (2026-07-08T06:18:38.563Z)**

```
275             return drawtext
276
277         def _select_encoder(self) -> Tuple[List[str], str]:
278             """Resolve the ``-c:v ?`` args used for EVERY per-clip trim re-encode this run,
plus the
279             chosen encoder name. Chosen ONCE (not per clip) so all clips share one codec and
the final
280             concat can stay ``-c copy``. NVENC (hardware) is the L4 win on the 4K re-encode;
if the
281             requested NVENC encoder can't run in this ffmpeg build (CI / non-GPU box), fall
back to
282             libx264 so a misconfig never nukes the reel."""
283             enc = self.encoder
284             if enc in ("h264_nvenc", "hevc_nvenc") and not _probe_encoder_available(enc):
285                 logger.warning(
286                     "[compile] encoder %r is not runnable in this ffmpeg build ? falling
back to "
287                     "libx264 (CPU). On the L4 worker this should be available.",
288                     enc,
289                 )
290                 enc = "libx264"
291             if enc == "libx264":
292                 # Unchanged historical CPU settings (byte-compatible with the pre-#521
stitch).
293                 return ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"], enc
294             # NVENC (H.264/HEVC): hardware encode. -cq is the CRF-equivalent constant-
quality knob; p4 is
295             # a balanced speed/quality preset. yuv420p keeps the output broadly playable.
296             return (
297                 ["-c:v", enc, "-preset", "p4", "-cq", "23", "-pix_fmt", "yuv420p"],
298                 enc,
299             )
300
```

```
301         def _trim_one_segment(
302             self,
303             idx: int,
304             raw_start: Union[int, float, str],
305             raw_end: Union[int, float, str],
306             segments: List[Tuple[float, float]],
307             video_duration: float,
308             tmp_dir: str,
309             raw_video_path: str,
310             watermark_filter: Optional[str],
311             video_codec_args: Optional[List[str]] = None,
312         ) -> Tuple[Optional[str], Optional[Tuple[int, float, float, str]]]:
313             """Trim a single segment into its own clip file (extracted verbatim from the
314             per-segment body of compile_highlights). Returns (clip_path, skip_record): on
```

**16. assistant (2026-07-08T06:18:41.976Z)**

Now let me read the existing tests to understand the test structure I'll extend.

**17. user (2026-07-08T06:18:46.047Z)**

```
1       """VideoCompiler coverage (previously zero ? the user-facing highlight-reel output path,
2       where the f-string SyntaxError lived undetected). ffmpeg/ffprobe are mocked."""
3
4       import os
5       import subprocess
6       from unittest.mock import MagicMock, patch
7
8       import pytest
9
10      from backend.pipeline.stitcher.compiler import VideoCompiler
11
12
13      def test_parse_time_formats():
14          f = VideoCompiler._parse_time
15          assert f(12.5) == 12.5
16          assert f("12.5") == 12.5
17          assert f("1:30") == 90.0
18          assert f("1:00:00") == 3600.0
19          assert f("garbage") == 0.0
20
21
22      def test_merge_overlapping_collapses_duplicates_and_overlaps():
23          m = VideoCompiler._merge_overlapping
24          # Exact duplicates + near-duplicates (the "every rally twice" re-run bug) collapse
to one each.
25          assert m([(28.2, 34.5), (28.5, 34.5), (40.5, 45.5), (40.5, 45.5)]) == [
26              (28.2, 34.5),
27              (40.5, 45.5),
28          ]
29          # Unsorted input is sorted; an overlapping range extends the previous.
30          assert m([(100.0, 110.0), (0.0, 5.0), (104.0, 120.0)]) == [
```

**18. user (2026-07-08T06:18:46.822Z)**

```
160             comp.compile_highlights(str(tmp_path / "nope.mp4"), [(0.0, 5.0)], "reel.mp4")
161
162
163     # ------------------------------------------------------------ encoder selection (#521
? NVENC)
164
165
166     def test_select_encoder_default_is_libx264(tmp_path):
167         """Default encoder = libx264 (CPU) with the historical settings ? no probe, works
```

```
everywhere."""
168         comp = VideoCompiler(str(tmp_path / "out"))
169         args, chosen = comp._select_encoder()
170         assert chosen == "libx264"
171         assert args == ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"]
172
173
174     def test_select_encoder_nvenc_when_available(tmp_path, monkeypatch):
175         """h264_nvenc resolves to the hardware encoder args when the probe reports it
runnable (the L4)."""
176         from backend.pipeline.stitcher import compiler as comp_mod
177
178         monkeypatch.setattr(comp_mod, "_probe_encoder_available", lambda enc: True)
179         comp = VideoCompiler(str(tmp_path / "out"), encoder="h264_nvenc")
180         args, chosen = comp._select_encoder()
181         assert chosen == "h264_nvenc"
182         assert args[:2] == ["-c:v", "h264_nvenc"]
183         assert "-cq" in args  # NVENC constant-quality knob (not -crf)
184
185
186     def test_select_encoder_falls_back_to_libx264_when_nvenc_unavailable(tmp_path,
monkeypatch):
187         """On a box where NVENC can't actually run (CI / no GPU), the requested encoder
falls back to
188         libx264 so a misconfig never nukes the reel."""
189         from backend.pipeline.stitcher import compiler as comp_mod
190
191         monkeypatch.setattr(comp_mod, "_probe_encoder_available", lambda enc: False)
192         comp = VideoCompiler(str(tmp_path / "out"), encoder="h264_nvenc")
193         args, chosen = comp._select_encoder()
194         assert chosen == "libx264"
195         assert args == ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"]
196
197
198     def test_compile_uses_selected_encoder_in_trim(tmp_path, monkeypatch):
199         """End-to-end (ffmpeg mocked): with an available NVENC encoder, every per-clip trim
command
200         carries -c:v h264_nvenc, and the concat stays -c copy."""
201         from backend.pipeline.stitcher import compiler as comp_mod
202
203         monkeypatch.setattr(comp_mod, "_probe_encoder_available", lambda enc: True)
204         comp = VideoCompiler(str(tmp_path / "out"), encoder="h264_nvenc")
205         raw = tmp_path / "raw.mp4"
206         raw.write_bytes(b"x")
207         trims = []
208
209         def fake_run(cmd, *a, **k):
210             if cmd[0] == "ffprobe":
211                 return MagicMock(stdout="100.0\n", returncode=0)
212             out = cmd[-1]
213             if isinstance(out, str) and out.endswith(".mp4"):
214                 open(out, "wb").write(b"clip")
215                 if "-i" in cmd and cmd[cmd.index("-i") + 1] == str(raw):
216                     trims.append(cmd)
217             return MagicMock(returncode=0, stdout=b"", stderr=b"")
218
219         with patch.object(subprocess, "run", side_effect=fake_run):
220             comp.compile_highlights(str(raw), [(0.0, 5.0), (10.0, 15.0)], "reel.mp4")
221         assert trims, "no trim commands captured"
222         for cmd in trims:
223             assert "h264_nvenc" in cmd  # hardware encode on every clip
```

**19. assistant (2026-07-08T06:19:13.615Z)**

Let me check the imports and module-level setup in the master compiler to design the test

correctly.

**20. user (2026-07-08T06:19:16.310Z)**

```
1       import logging
2       import os
3       import subprocess
4       import tempfile
5       from pathlib import Path
6       from typing import Any, Dict, List, Optional, Tuple, Union
7
8       from backend.utils.ffmpeg import (  # P2a: single ffmpeg/ffprobe resolver
9           ffmpeg_bin,
10          ffmpeg_timeout_sec,
11          ffprobe_bin,
12          ffprobe_timeout_sec,
13          timeout_label,
14      )
15
16      logger = logging.getLogger(__name__)
17
18      # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the
overlay
19      # renders out-of-the-box without a system fontconfig setup (a common gap on Windows,
where
20      # `font='Sans'` fails). Overridable via stitching.watermark.font_path. This module lives
at
21      # backend/pipeline/stitcher/, so parents[2] == backend/.
22      _BUNDLED_FONT = (
23          Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-Regular.ttf"
24      )
```

**21. assistant (2026-07-08T06:19:55.957Z)**

I now fully understand the bug and the fix. The buggy line is `color=c=black:s=64x64:d=0.1` (line 51), and all existing encoder tests monkeypatch `_probe_encoder_available` wholesale, so none exercise the real probe command geometry ? that's the regression gap I'll close.

Given the dirty tree (conflicted sibling WIP in my exact target file), I'll do this work in an **isolated git worktree** off `origin/master` so I never touch that WIP. Let me set it up.

**22. user (2026-07-08T06:20:02.000Z)**

```
Preparing worktree (new branch 'fix/nvenc-probe-resolution')
branch 'fix/nvenc-probe-resolution' set up to track 'origin/master'.
Updating files:  57% (319/552)
Updating files:  58% (321/552)
Updating files:  59% (326/552)
Updating files:  60% (332/552)
Updating files:  61% (337/552)
Updating files:  62% (343/552)
Updating files:  63% (348/552)
Updating files:  64% (354/552)
Updating files:  65% (359/552)
Updating files:  66% (365/552)
Updating files:  67% (370/552)
Updating files:  68% (376/552)
Updating files:  69% (381/552)
Updating files:  70% (387/552)
Updating files:  71% (392/552)
Updating files:  72% (398/552)
Updating files:  73% (403/552)
Updating files:  74% (409/552)
Updating files:  75% (414/552)
```

```
Updating files:  76% (420/552)
Updating files:  77% (426/552)
Updating files:  78% (431/552)
Updating files:  79% (437/552)
Updating files:  80% (442/552)
Updating files:  81% (448/552)
Updating files:  82% (453/552)
Updating files:  83% (459/552)
Updating files:  84% (464/552)
Updating files:  85% (470/552)
Updating files:  86% (475/552)
Updating files:  87% (481/552)
Updating files:  88% (486/552)
Updating files:  89% (492/552)
Updating files:  90% (497/552)
Updating files:  91% (503/552)
Updating files:  92% (508/552)
Updating files:  93% (514/552)
Updating files:  94% (519/552)
Updating files:  94% (524/552)
Updating files:  95% (525/552)
Updating files:  96% (530/552)
Updating files:  97% (536/552)
Updating files:  98% (541/552)
Updating files:  99% (547/552)
Updating files: 100% (552/552)
Updating files: 100% (552/552), done.
HEAD is now at dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven
phase?machine placement (#521) (#525)
```

**23. assistant (2026-07-08T06:20:08.447Z)**

```
Worktree created on `fix/nvenc-probe-resolution` off `origin/master` (dac6736), fully isolated
from the conflicted WIP. Let me verify its state and confirm the target file is clean there.
```

**24. user (2026-07-08T06:20:12.863Z)**

```
On branch fix/nvenc-probe-resolution
Your branch is up to date with 'origin/master'.

nothing to commit, working tree clean
===branch/HEAD===
dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521) (#525)
===commits vs origin/master (must be empty)===
```

**25. assistant (2026-07-08T06:20:18.423Z)**

```
Clean worktree, correct branch, zero extra commits. Now let me read the actual target file in
the worktree and apply the fix.
```

**26. user (2026-07-08T06:20:19.289Z)**

```
31      def _probe_encoder_available(encoder: str) -> bool:
32          """Can this ffmpeg build actually RUN ``encoder`` (not merely list it)? Runs a tiny
1-frame
33          lavfi test encode to null. NVENC is often *compiled in* on boxes without an NVIDIA
GPU, so a
34          real test encode ? not ``-encoders`` grep ? is what distinguishes "usable" from
"will fail at
35          runtime". Memoized. Any error / timeout ? unavailable (caller falls back to
libx264)."""
36          if encoder in _ENCODER_AVAILABLE:
37              return _ENCODER_AVAILABLE[encoder]
```

```
38          ok = False
39          # The probe is a tiny 1-frame encode ? bound it to a short timeout (? the configured
ffmpeg
40          # budget, or 30 s when unbounded) so a wedged encoder can't hang the whole stitch.
41          configured = ffmpeg_timeout_sec()
42          probe_timeout = min(30.0, float(configured)) if configured else 30.0
43          try:
44              proc = subprocess.run(
45                  [
46                      ffmpeg_bin(),
47                      "-hide_banner",
48                      "-f",
49                      "lavfi",
50                      "-i",
51                      "color=c=black:s=64x64:d=0.1",
52                      "-frames:v",
53                      "1",
54                      "-c:v",
55                      encoder,
56                      "-f",
57                      "null",
58                      "-",
59                  ],
60                  capture_output=True,
61                  timeout=probe_timeout,
62              )
63              ok = proc.returncode == 0
64          except Exception:  # noqa: BLE001 ? a probe failure/timeout just means "not usable ?
fall back"
65              ok = False
66          _ENCODER_AVAILABLE[encoder] = ok
67          return ok
```

**27. assistant (2026-07-08T06:20:50.182Z)**

The fix is a one-value change, but I'll make it durable: introduce a well-documented module constant so nobody "optimizes" the probe back below NVENC's minimum, and encode the *why* (the empirical characterization) right at the site. Let me apply the edits.

**28. user (2026-07-08T06:20:58.722Z)**

The file C:/Users/avidu/Projects/khelsutra-guru/bhi-nvenc-probe-wt/backend/pipeline/stitcher/compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

**29. user (2026-07-08T06:21:00.788Z)**

The file C:/Users/avidu/Projects/khelsutra-guru/bhi-nvenc-probe-wt/backend/pipeline/stitcher/compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

**30. assistant (2026-07-08T06:21:06.056Z)**

Now the test. Let me read the end of the worktree's test file to find the right insertion point after the existing encoder tests.

**31. user (2026-07-08T06:21:06.829Z)**

```
196
197
198     def test_compile_uses_selected_encoder_in_trim(tmp_path, monkeypatch):
199         """End-to-end (ffmpeg mocked): with an available NVENC encoder, every per-clip trim
command
200         carries -c:v h264_nvenc, and the concat stays -c copy."""
```

```
201         from backend.pipeline.stitcher import compiler as comp_mod
202
203         monkeypatch.setattr(comp_mod, "_probe_encoder_available", lambda enc: True)
204         comp = VideoCompiler(str(tmp_path / "out"), encoder="h264_nvenc")
205         raw = tmp_path / "raw.mp4"
206         raw.write_bytes(b"x")
207         trims = []
208
209         def fake_run(cmd, *a, **k):
210             if cmd[0] == "ffprobe":
211                 return MagicMock(stdout="100.0\n", returncode=0)
212             out = cmd[-1]
213             if isinstance(out, str) and out.endswith(".mp4"):
214                 open(out, "wb").write(b"clip")
215                 if "-i" in cmd and cmd[cmd.index("-i") + 1] == str(raw):
216                     trims.append(cmd)
217             return MagicMock(returncode=0, stdout=b"", stderr=b"")
218
219         with patch.object(subprocess, "run", side_effect=fake_run):
220             comp.compile_highlights(str(raw), [(0.0, 5.0), (10.0, 15.0)], "reel.mp4")
221         assert trims, "no trim commands captured"
222         for cmd in trims:
223             assert "h264_nvenc" in cmd  # hardware encode on every clip
224             assert "libx264" not in cmd
```

**32. assistant (2026-07-08T06:21:41.890Z)**

The test needs to exercise the *real* probe (existing tests all mock it away), so I must be sure `ffmpeg_bin()`/`ffmpeg_timeout_sec()` won't raise when the real function runs under mocked subprocess. Let me check that helper.

**33. user (2026-07-08T06:21:43.883Z)**

```
32:def ffmpeg_bin() -> str:
33-    """The ffmpeg executable: ``RALLY_FFMPEG`` if set, else ``"ffmpeg"`` (resolved on
PATH)."""
34-    return (os.environ.get("RALLY_FFMPEG") or "").strip() or "ffmpeg"
35-
36-
37-def ffprobe_bin() -> str:
38-    """The ffprobe executable: ``RALLY_FFPROBE`` if set, else ``"ffprobe"`` (resolved on
PATH)."""
39-    return (os.environ.get("RALLY_FFPROBE") or "").strip() or "ffprobe"
40-
41-
42-def _timeout_seconds(env_var: str, default: float) -> Optional[float]:
43-    raw = os.environ.get(env_var)
44-    if raw is None or raw.strip() == "":
45-        value = default
46-    else:
47-        try:
48-            value = float(raw)
49-        except ValueError:
50-            logger.warning("%s=%r is not numeric; using %.0fs", env_var, raw, default)
--
62:def ffmpeg_timeout_sec() -> Optional[float]:
63-    """Bound transcode/extract/concat calls. Set ``RALLY_FFMPEG_TIMEOUT_SEC=0`` to wait
forever."""
64-    return _timeout_seconds("RALLY_FFMPEG_TIMEOUT_SEC", DEFAULT_FFMPEG_TIMEOUT_SEC)
65-
66-
67-def timeout_label(seconds: Optional[float]) -> str:
68-    return "forever" if seconds is None else f"{seconds:g}s"
69-
70-
```

```
71-def ffmpeg_available() -> bool:
72-    return shutil.which(ffmpeg_bin()) is not None
73-
74-
75-def ffprobe_available() -> bool:
76-    return shutil.which(ffprobe_bin()) is not None
77-
78-
79-def install_hint() -> str:
80-    """The OS-appropriate one-line install command (mirrors scripts/doctor.py)."""
```

**34. assistant (2026-07-08T06:21:53.851Z)**

`ffmpeg_bin()` just returns a string and never raises ? safe to exercise the real probe under mocked `subprocess.run`. Now let me add the tests. First the `re` import, then the regression-guard tests appended after the existing encoder tests.

**35. user (2026-07-08T06:21:55.600Z)**

The file C:/Users/avidu/Projects/khelsutra-guru/bhi-nvenc-probe-wt/tests/test_compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

**36. user (2026-07-08T06:22:15.315Z)**

The file C:/Users/avidu/Projects/khelsutra-guru/bhi-nvenc-probe-wt/tests/test_compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

**37. assistant (2026-07-08T06:22:32.532Z)**

Now let me run the tests. First, let me understand the test setup (Python, venv, conftest) in the worktree so imports resolve cleanly.

**38. user (2026-07-08T06:22:38.495Z)**

```
Exit code 2
Python 3.13.13
---venv?---
no venv dir in main repo
---conftest/pytest cfg (worktree)---
C:/Users/avidu/Projects/khelsutra-guru/bhi-nvenc-probe-wt/pyproject.toml
C:/Users/avidu/Projects/khelsutra-guru/bhi-nvenc-probe-wt/pytest.ini
```

**39. user (2026-07-08T06:22:50.951Z)**

```
1       [pytest]
2       # When `pytest` is invoked without an explicit path, only collect from tests/.
3       # scratch/ holds debugging scripts that hit real APIs (Gemini upload, etc.)
4       # and should never run as part of the regular test suite.
5       testpaths = tests
6
```

**40. assistant (2026-07-08T06:23:12.720Z)**

No `pythonpath` override ? so imports resolve from cwd. Critical: the Bash tool's cwd is the **main repo** (with the conflicted WIP), so I must run pytest *inside the worktree* or `import backend` would pull the main repo's conflicted code. I'll use a subshell `cd` into the worktree.

**41. user (2026-07-08T06:23:21.574Z)**

```
...............                                                          [100%]
15 passed in 1.06s
```

**42. assistant (2026-07-08T06:23:31.356Z)**

All 15 pass (12 original + 3 new). Let me confirm my new tests are actually collected and that
the regression guard genuinely fails against the old `64x64` value (proving it guards the bug).

**43. user (2026-07-08T06:23:37.907Z)**

```
============================= test session starts =============================
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
rootdir: C:\Users\avidu\Projects\khelsutra-guru\bhi-nvenc-probe-wt
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 15 items / 12 deselected / 3 selected

tests/test_compiler.py::test_probe_encoder_uses_safe_resolution PASSED   [ 33%]
tests/test_compiler.py::test_probe_encoder_builds_single_frame_null_encode PASSED [ 66%]
tests/test_compiler.py::test_probe_encoder_returns_false_on_ffmpeg_failure PASSED [100%]

====================== 3 passed, 12 deselected in 0.81s =======================
```

**44. user (2026-07-08T06:23:39.105Z)**

```
    64x64 -> guard FAIL (correctly rejected)
  256x256 -> guard PASS
```

**45. assistant (2026-07-08T06:23:46.465Z)**

Both new tests pass and the guard provably rejects the old `64x64` while accepting `256x256`.
Now let me honor the repo's verification rigor: full suite, lint, and type-check. Let me run the
full offline suite first.

**46. user (2026-07-08T06:25:47.482Z)**

```
Exit code 143
Command timed out after 2m 0s
```

**47. assistant (2026-07-08T06:25:55.300Z)**

The suite is large (timed out at 2m). Let me see what `run_all_tests.py` actually runs so I
invoke it efficiently.

**48. user (2026-07-08T06:25:55.653Z)**

```
1       #!/usr/bin/env python
2       """Run the FULL test suite for the badminton-highlight-indexer.
3
4       A single, discoverable entry point so you (or a reviewer / CI) can verify the
5       whole suite before opening a PR:
6
7           python run_all_tests.py             # full suite, quiet
8           python run_all_tests.py -v          # verbose (passthrough to pytest)
9           python run_all_tests.py -k config   # filter by keyword (passthrough)
10          python run_all_tests.py --cov=backend --cov-report=term-missing   # coverage
11
12      Anything after the script name is passed straight through to pytest.
13
14      Notes
15      -----
16      - The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
17      - The observability **reporting** tests skip cleanly if the optional ``obsreport``
18        package isn't installed (they use ``pytest.importorskip``). To exercise them too:
19            pip install -e ../sports-obsreport --no-build-isolation
20      - Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).
21      """
22
```

```
23      from __future__ import annotations
24
25      import importlib.util
26      import subprocess
27      import sys
28
29
30      def _installed(module: str) -> bool:
31          try:
32              return importlib.util.find_spec(module) is not None
33          except (ImportError, ValueError):
34              return False
35
36
37      def main() -> int:
38          args = list(sys.argv[1:])
39          # Default to quiet unless the caller chose a verbosity.
40          if not any(a in ("-q", "-v", "-vv", "-vvv", "--quiet", "--verbose") for a in args):
41              args = ["-q", *args]
42          cmd = [sys.executable, "-m", "pytest", *args]
43
44          obs = (
45              "available"
46              if _installed("obsreport")
47              else "NOT installed ? reporting tests will SKIP"
48          )
49          bar = "=" * 72
50          print(bar)
51          print(" Full test suite:  " + " ".join(cmd))
52          print(f" obsreport (optional reporting dep): {obs}")
53          print(bar, flush=True)
54
55          rc = subprocess.run(cmd).returncode
56
57          print(bar)
58          print(
59              " RESULT: "
60              + ("PASS - all tests green" if rc == 0 else f"FAIL - pytest exit {rc}")
61          )
62          print(bar)
63          return rc
64
65
66      if __name__ == "__main__":
67          raise SystemExit(main())
68
```

**49. assistant (2026-07-08T06:26:06.529Z)**

`run_all_tests.py` is just a pytest wrapper. Let me run the full suite in the **background** (it needs >2min, no xdist available) and meanwhile run lint + type checks in the foreground.

**50. user (2026-07-08T06:26:08.243Z)**

Command running in background with ID: b15vqbc2k. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\b55cb444-5f5e-4770-ad71-cee76683ff71\tasks\b15vqbc2k.output. You will be notified when it completes. To check interim output, use Read on that file path.

**51. user (2026-07-08T06:26:11.369Z)**

```
=== ruff (changed files) ===
All checks passed!
(ruff exit: 0)
```

**52. assistant (2026-07-08T06:26:19.413Z)**

Ruff clean. Now let me match the repo's CI for mypy ? let me find how CI invokes it.

**53. user (2026-07-08T06:26:22.715Z)**

```
=== workflow mypy/ruff lines ===
.github/workflows/ci.yml:16:  # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle +
bugbear.
.github/workflows/ci.yml:17:  # Also the repo-wide syntax guard ? ruff reports invalid-syntax on
every
.github/workflows/ci.yml:29:      - name: Install ruff
.github/workflows/ci.yml:30:        # Pinned so an upstream ruff release can't redden every PR
overnight; bump deliberately.
.github/workflows/ci.yml:31:        run: python -m pip install ruff==0.15.16
.github/workflows/ci.yml:33:        run: ruff check . --output-format github
.github/workflows/ci.yml:66:  # Type gate (mypy.ini at repo root): lenient green baseline with
an explicit
.github/workflows/ci.yml:68:  # up. Typed core deps are installed so mypy sees their real
signatures
.github/workflows/ci.yml:79:      - name: Install mypy + typed core deps
.github/workflows/ci.yml:80:        # mypy pinned alongside ruff so a type-checker release can't
redden CI overnight.
.github/workflows/ci.yml:81:        run: python -m pip install mypy==2.1.0 pydantic fastapi
uvicorn python-dotenv google-genai numpy
.github/workflows/ci.yml:83:        run: mypy backend training
.github/workflows/ci.yml:136:        # (pytest, pytest-cov, httpx for the fastapi TestClient,
ruff, mypy).
.github/workflows/nightly.yml:43:  # Lint gate (ruff.toml at repo root) ? mirrors ci.yml::lint.
.github/workflows/nightly.yml:53:      - name: Install ruff
.github/workflows/nightly.yml:54:        # Pinned so an upstream ruff release can't redden the
nightly overnight; bump deliberately.
.github/workflows/nightly.yml:55:        run: python -m pip install ruff==0.15.16
.github/workflows/nightly.yml:57:        run: ruff check . --output-format github
.github/workflows/nightly.yml:74:  # Type gate (mypy.ini at repo root) ? mirrors
ci.yml::typecheck. Typed core
.github/workflows/nightly.yml:75:  # deps are installed so mypy sees real signatures (without
them
.github/workflows/nightly.yml:86:      - name: Install mypy + typed core deps
.github/workflows/nightly.yml:87:        # mypy pinned alongside ruff so a type-checker release
can't redden the nightly overnight.
.github/workflows/nightly.yml:88:        run: python -m pip install mypy==2.1.0 pydantic fastapi
uvicorn python-dotenv google-genai numpy
.github/workflows/nightly.yml:90:        run: mypy backend training
.github/workflows/nightly.yml:255:              'One of ruff / leak-scan / mypy / import-smoke /
full-suite (cov>=70) /',
=== mypy config in pyproject ===
=== mypy.ini? ===
mypy.ini
none
```

**54. assistant (2026-07-08T06:26:37.985Z)**

CI runs `ruff check .` and `mypy backend training` (mypy.ini config). Let me run the full-repo ruff in the worktree (clean base, so it should be green) and check mypy availability.

**55. user (2026-07-08T06:26:42.270Z)**

```
=== ruff check . ===
All checks passed!
(exit 0)
=== mypy version ===
mypy 2.1.0 (compiled: yes)
```

**56. assistant (2026-07-08T06:26:48.038Z)**

`ruff check .` is fully green and mypy is exactly 2.1.0 (CI-matching). Now the type gate exactly as CI runs it: `mypy backend training`.

**57. user (2026-07-08T06:28:02.579Z)**

```
backend\annotations\base.py:88: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\sports\__init__.py:9: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\pipeline\validators\base.py:123: note: By default the bodies of untyped functions are
not checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 150 source files
(mypy exit: 0)
```

**58. assistant (2026-07-08T06:28:14.880Z)**

mypy is green (exit 0, "no issues found in 150 source files"; only informational notes). Let me check the full-suite progress.

**59. user (2026-07-08T06:28:17.033Z)**

<system-reminder>Warning: the file exists but is shorter than the provided offset (1). The file has 1 lines.</system-reminder>

**60. assistant (2026-07-08T06:28:28.316Z)**

The background suite buffers output until it finishes (I'll be notified). While it runs, let me review the exact diff I'm about to commit.

**61. user (2026-07-08T06:28:31.041Z)**

```
 backend/pipeline/stitcher/compiler.py | 19 ++++++++---
 tests/test_compiler.py                | 62 +++++++++++++++++++++++++++++++++++++
 2 files changed, 77 insertions(+), 4 deletions(-)
========== FULL DIFF ==========
diff --git a/backend/pipeline/stitcher/compiler.py b/backend/pipeline/stitcher/compiler.py
index 5c7229e..3789cfa 100644
--- a/backend/pipeline/stitcher/compiler.py
+++ b/backend/pipeline/stitcher/compiler.py
@@ -27,12 +27,23 @@ _BUNDLED_FONT = (
 #: it and the rest reuse the verdict. Keyed by encoder name; cleared only across processes.
 _ENCODER_AVAILABLE: Dict[str, bool] = {}

+#: Frame geometry for the 1-frame NVENC probe encode in ``_probe_encoder_available``. It MUST stay
+#: at or above NVENC's minimum encode dimensions: on real GPUs the hardware H.264/HEVC encoders
+#: REJECT tiny frames, so a sub-minimum probe (the original 64x64) FALSELY reports NVENC as
+#: unavailable and ``_select_encoder`` silently falls back to libx264 ? defeating the NVENC-on-L4
+#: re-encode win of #521/#525. Empirically (RTX 2070 Super, ffmpeg 8.1.1) h264_nvenc rejects
+#: 64x64 and 128x128 but accepts ~145px and up; 256x256 keeps a safe margin. Do NOT shrink below
+#: 256x256 ? guarded by tests/test_compiler.py::test_probe_encoder_uses_safe_resolution.
+_PROBE_SOURCE_SIZE = "256x256"
+

 def _probe_encoder_available(encoder: str) -> bool:
     """Can this ffmpeg build actually RUN ``encoder`` (not merely list it)? Runs a tiny 1-frame
-    lavfi test encode to null. NVENC is often *compiled in* on boxes without an NVIDIA GPU, so a
+
```

```
-    real test encode ? not ``-encoders`` grep ? is what distinguishes "usable" from "will fail
at
-    runtime". Memoized. Any error / timeout ? unavailable (caller falls back to libx264)."""
+    lavfi test encode to null at ``_PROBE_SOURCE_SIZE`` (a resolution ? NVENC's minimum, so the
+    probe reflects real hardware runnability rather than rejecting an undersized frame). NVENC
is
+    often *compiled in* on boxes without an NVIDIA GPU, so a real test encode ? not
``-encoders``
+    grep ? is what distinguishes "usable" from "will fail at runtime". Memoized. Any error /
+    timeout ? unavailable (caller falls back to libx264)."""
     if encoder in _ENCODER_AVAILABLE:
         return _ENCODER_AVAILABLE[encoder]
     ok = False
@@ -48,7 +59,7 @@ def _probe_encoder_available(encoder: str) -> bool:
                 "-f",
                 "lavfi",
                 "-i",
-                "color=c=black:s=64x64:d=0.1",
+                f"color=c=black:s={_PROBE_SOURCE_SIZE}:d=0.1",
                 "-frames:v",
                 "1",
                 "-c:v",
diff --git a/tests/test_compiler.py b/tests/test_compiler.py
index f3f2ae9..226febd 100644
--- a/tests/test_compiler.py
+++ b/tests/test_compiler.py
@@ -2,6 +2,7 @@
 where the f-string SyntaxError lived undetected). ffmpeg/ffprobe are mocked."""

 import os
+import re
 import subprocess
 from unittest.mock import MagicMock, patch

@@ -222,3 +223,64 @@ def test_compile_uses_selected_encoder_in_trim(tmp_path, monkeypatch):
     for cmd in trims:
         assert "h264_nvenc" in cmd  # hardware encode on every clip
         assert "libx264" not in cmd
+
+
+# ------------------------------------------------- NVENC probe geometry (#521/#525
regression guard)
+
+
+def _capture_probe_cmd(monkeypatch):
+    """Run the REAL _probe_encoder_available with subprocess.run mocked; return the ffmpeg argv
it
+    built. Resets the cross-call memo so the command is actually constructed (not short-
circuited by
+    a cached verdict), and mocks subprocess so no real ffmpeg is needed."""
+    from backend.pipeline.stitcher import compiler as comp_mod
+
+    monkeypatch.setattr(comp_mod, "_ENCODER_AVAILABLE", {})
+    captured = {}
+
+    def fake_run(cmd, *a, **k):
+        captured["cmd"] = list(cmd)
+        return MagicMock(returncode=0)
+
+    monkeypatch.setattr(comp_mod.subprocess, "run", fake_run)
+    assert comp_mod._probe_encoder_available("h264_nvenc") is True
+    assert "cmd" in captured, "probe never invoked subprocess.run"
+    return captured["cmd"]
+
+
```

```
+def test_probe_encoder_uses_safe_resolution(monkeypatch):
+    """The 1-frame NVENC probe encode MUST use a source resolution >= NVENC's minimum encode
+    dimensions (>=256px per axis). The original 64x64 is BELOW that minimum, so on real GPUs
the
+    probe FAILS ("at least one of its streams received no packets") even though NVENC works
fine at
+    real resolutions ? making _select_encoder silently fall back to libx264 and defeating the
+    NVENC-on-L4 re-encode win of #521/#525. Guard the geometry against regressing to a sub-
minimum
+    size."""
+    cmd = _capture_probe_cmd(monkeypatch)
+    source = cmd[cmd.index("-i") + 1]
+    match = re.search(r"s=(\d+)x(\d+)", source)
+    assert match, f"probe lavfi source has no s=WxH size spec: {source!r}"
+    width, height = int(match.group(1)), int(match.group(2))
+    assert width >= 256 and height >= 256, (
+        f"probe resolution {width}x{height} is below NVENC's minimum; must be >=256x256 so the
"
+        "probe reflects real NVENC runnability instead of rejecting an undersized frame
(#521/#525)"
+    )
+
+
+def test_probe_encoder_builds_single_frame_null_encode(monkeypatch):
+    """Sanity: the probe stays a real 1-frame test encode to null with the requested codec (not
an
+    ``-encoders`` capability grep), so it still distinguishes 'NVENC compiled-in but unusable'
from
+    genuinely usable."""
+    cmd = _capture_probe_cmd(monkeypatch)
+    assert cmd[cmd.index("-c:v") + 1] == "h264_nvenc"  # probes the requested encoder
+    assert cmd[cmd.index("-frames:v") + 1] == "1"  # exactly one frame
+    assert "null" in cmd  # encodes to the null muxer, discarding output
+    assert "-encoders" not in cmd  # a real encode, not a capability grep
+
+
+def test_probe_encoder_returns_false_on_ffmpeg_failure(monkeypatch):
+    """A non-zero ffmpeg exit (the encoder can't run on this box) ? unavailable ? caller falls
back
+    to libx264. Previously untested: the existing encoder tests monkeypatch the whole probe
away."""
+    from backend.pipeline.stitcher import compiler as comp_mod
+
+    monkeypatch.setattr(comp_mod, "_ENCODER_AVAILABLE", {})
+    monkeypatch.setattr(comp_mod.subprocess, "run", lambda *a, **k: MagicMock(returncode=1))
+    assert comp_mod._probe_encoder_available("h264_nvenc") is False
```

#### 62. assistant (2026-07-08T06:28:47.503Z)

The diff is clean and complete. While the full suite runs, let me confirm the PR tooling is ready (remote + `gh` auth).

#### 63. user (2026-07-08T06:28:53.104Z)

```
=== remotes ===
origin  https://github.com/Khelsutra/badminton-highlight-indexer.git (fetch)
origin  https://github.com/Khelsutra/badminton-highlight-indexer.git (push)
=== gh auth ===
github.com
  ? Logged in to github.com account avidullu (keyring)
  - Active account: true
  - Git operations protocol: https
  - Token: gho_**********************************
  - Token scopes: 'gist', 'read:org', 'repo', 'workflow'
=== default branch ===
```

master

**64. assistant (2026-07-08T06:28:57.955Z)**

Everything's ready for the PR (origin ? Khelsutra/badminton-highlight-indexer, `gh` authed, default branch `master`). Let me check on the full suite.

**65. user (2026-07-08T06:29:01.826Z)**

(Bash completed with no output)

**66. assistant (2026-07-08T06:29:23.738Z)**

The background suite buffers until completion (I'll be notified). Let me use the wait productively by drafting the commit message and PR body to scratchpad so I can act immediately once it's green.

**67. assistant (2026-07-08T06:29:37.408Z)**

fix(serving): probe NVENC at 256x256, not 64x64, so it isn't falsely rejected (#521/#525)

`_probe_encoder_available` (added in #521, merged via #525) ran its 1-frame test encode from `color=c=black:s=64x64:d=0.1`. 64x64 is BELOW NVENC's minimum encode dimensions, so on a real GPU the probe fails ("at least one of its streams received no packets. Conversion failed!") even though the hardware encoder works fine at real resolutions.

Consequence: on any GPU box whose NVENC minimum exceeds 64x64 ? potentially including the L4 Cloud Run worker ? `_select_encoder` silently falls back to libx264, so the 4K watermark re-encode stays on slow CPU and #521's NVENC-on-L4 win is defeated. It went unnoticed because the 2026-07-08 CUJ that motivated #521 exercised the control-plane (libx264) path, not a live L4 NVENC stitch.

Empirically characterized on an RTX 2070 Super (ffmpeg 8.1.1): h264_nvenc probe FAILs at 64x64 and 128x128 but PASSes at ~145px and up (256x256, 320x240 all pass; 4K encode works). Same failures with a real testsrc2 source, so it's NVENC's floor, not a `color`-source artifact.

Fix: bump the probe source to a module constant `_PROBE_SOURCE_SIZE = "256x256"` (a safe margin above the observed floor) and document why it must never shrink below the NVENC minimum. It stays a real 1-frame encode to null (not an `-encoders` grep), so it still distinguishes compiled-in-but-unusable NVENC from genuinely usable.

Tests (tests/test_compiler.py): the existing encoder tests all monkeypatch the whole probe away, so the real command geometry was never exercised. Add `test_probe_encoder_uses_safe_resolution` (parses the built argv and asserts s=WxH is >=256 on both axes ? guards against regressing to a sub-minimum size), plus companions asserting it stays a single-frame null encode of the requested codec and returns False on a non-zero ffmpeg exit.

Verified locally: full offline suite green (`python run_all_tests.py`), `ruff check .` clean, `mypy backend training` clean (mypy 2.1.0, CI-pinned). Live GPU probe PASS at 256x256 to be confirmed by the owner on a real NVENC box.

Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>

**68. user (2026-07-08T06:29:37.855Z)**

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer/b55cb444-5f5e-4770-ad71-cee76683ff71/scratchpad/commit_msg.txt (file state is current in your context ? no need to Read it back)

**69. assistant (2026-07-08T06:29:53.966Z)**

I've completed the implementation and the fast verification gates. Here's where things stand while the full suite finishes:

**Done (in an isolated worktree off `origin/master`):**
- **Fix** ? [backend/pipeline/stitcher/compiler.py](backend/pipeline/stitcher/compiler.py): the probe now encodes from a `_PROBE_SOURCE_SIZE = "256x256"` constant instead of the sub-minimum `64x64`, with a comment documenting the NVENC-floor reasoning (RTX 2070S/ffmpeg 8.1.1 characterization) so it can't be "optimized" back down.
- **Tests** ? [tests/test_compiler.py](tests/test_compiler.py): added `test_probe_encoder_uses_safe_resolution` (parses the built ffmpeg argv, asserts `s=WxH` ? 256 on both axes ? the regression guard you asked for), plus two companions (single-frame null-encode shape; `False` on non-zero exit). The existing tests all monkeypatched the probe away, so the real command geometry was previously never exercised.

**Verified green:** targeted compiler tests (15 passed), and I confirmed the new guard *fails* against `64x64` and *passes* at `256x256`. `ruff check .` clean, `mypy backend training` clean (mypy 2.1.0, exactly CI's pin).

**Why a worktree:** your working tree has conflicted sibling WIP (`UU` marks on `orchestration.py` and `compiler.py`, a `sd-reel-engine-wip` stash, branch `codex/issue-523-sd-reel-plan`) ? no merge actually in progress, but my target file is mid-conflict. I isolated my change in a separate worktree so I never touched that WIP.

The full offline suite (`run_all_tests.py`) is running in the background ? it needs >2 min and there's no `xdist` to parallelize. I'll commit, push `fix/nvenc-probe-resolution`, and open the PR referencing #521/#525 once it's green. Waiting on that result now.

**70. user (2026-07-08T06:31:19.444Z)**

<task-notification>
<task-id>b15vqbc2k</task-id>
<tool-use-id>toolu_01RHx213D7B7yJdigPPYxh2C</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\b55cb444-5f5e-4770-ad71-cee76683ff71\tasks\b15vqbc2k.output</output-file>
<status>completed</status>
<summary>Background command "Run full offline test suite (background)" completed (exit code 0)</summary>
</task-notification>

**71. assistant (2026-07-08T06:31:22.126Z)**

Full suite completed with **exit code 0**. Let me confirm the result summary.

**72. user (2026-07-08T06:31:30.347Z)**

```
.................................................................... [ 91%]
.........................s.......................................... [ 95%]
.................................................................... [ 99%]
..............                                                       [100%]
============================== warnings summary ===============================
tests/test_api_cloud_dispatch.py: 8 warnings
tests/test_api_endpoints.py: 10 warnings
  C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end user credentials from Google Cloud SDK without a quota project. You might receive a "quota exceeded" or "API not enabled" error. See the following page for troubleshooting: https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
    warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
1878 passed, 8 skipped, 18 warnings in 300.38s (0:05:00)
```

```
======================================================================
 RESULT: PASS - all tests green
======================================================================
```

**73. assistant (2026-07-08T06:31:47.383Z)**

Full suite green: **1878 passed, 8 skipped** (clean optional-dep skips), exit 0. All gates pass ? suite, `ruff check .`, and `mypy backend training`. Now let me commit, following the branching-safety protocol (re-verify HEAD, stage explicit paths only).

**74. user (2026-07-08T06:31:53.343Z)**

```
=== branch/HEAD (must be mine) ===
fix/nvenc-probe-resolution
dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521) (#525)
=== staging explicit paths ===
M  backend/pipeline/stitcher/compiler.py
M  tests/test_compiler.py
```

**75. assistant (2026-07-08T06:31:57.607Z)**

Only my two files staged, on the right branch. Committing with the prepared message.

**76. user (2026-07-08T06:32:05.328Z)**

```
[fix/nvenc-probe-resolution 7d6c049] fix(serving): probe NVENC at 256x256, not 64x64, so it
isn't falsely rejected (#521/#525)
 2 files changed, 77 insertions(+), 4 deletions(-)
=== verify: only my commit vs origin/master ===
7d6c049 fix(serving): probe NVENC at 256x256, not 64x64, so it isn't falsely rejected
(#521/#525)
```

**77. assistant (2026-07-08T06:32:10.947Z)**

Committed as `7d6c049`, and `origin/master..HEAD` shows exactly one commit ? mine, no sibling contamination. Now let me push and open the PR referencing #521/#525.

**78. user (2026-07-08T06:32:16.499Z)**

```
remote:
remote: Create a pull request for 'fix/nvenc-probe-resolution' on GitHub by visiting:
remote:      https://github.com/Khelsutra/badminton-highlight-indexer/pull/new/fix/nvenc-probe-
resolution
remote:
branch 'fix/nvenc-probe-resolution' set up to track 'origin/fix/nvenc-probe-resolution'.
To https://github.com/Khelsutra/badminton-highlight-indexer.git
 * [new branch]      fix/nvenc-probe-resolution -> fix/nvenc-probe-resolution
```

**79. assistant (2026-07-08T06:32:23.265Z)**

Pushed. Let me check the PR template so I fill any required sections, then open the PR.

**80. user (2026-07-08T06:32:28.110Z)**

```
<!-- Describe the change and link the issue it closes. -->
```

## Summary

## Checks

- [ ] `python run_all_tests.py` green; `ruff check .` + `mypy` clean; coverage not lowered.

- [ ] Internal markdown links valid (`python -m tools.check_md_links`).

## Launch / "bit-flip" PRs only

A PR that flips a default / "launch bit" so a tracked project reaches terminal state MUST also run
the `docs/DOC_STATUS.md` §2 archival due-diligence on that project's docs, **in this same PR**:
- [ ] Verified each doc's claims against the code (tagged load-bearing facts `[verified]`).
- [ ] Flipped the doc Status ? DELIVERED/DONE/REJECTED + a completion note (what shipped, PR, paths).
- [ ] Updated all inbound references (`docs/README.md`, `CLAUDE.md`, `docs/NEXT_STEPS.md`, cross-doc links).
- [ ] Kept the ADR forward/back lineage.
- [ ] `git mv` the terminal docs to `docs/archives/?` and recorded the move in `DOC_STATUS.md` §3.
- [ ] Fixed any internal-link breakage from the move (`link-check` CI job green).

_(Not a launch/bit-flip PR? Delete this section.)_

**81. assistant (2026-07-08T06:32:40.086Z)**

Pure bugfix ? not a launch/bit-flip PR, and it touches zero markdown, so the doc-archival and link-check sections are N/A. Let me write the PR body and open it against `master`.

**82. assistant (2026-07-08T06:32:56.585Z)**

## Summary

`_probe_encoder_available` (added in #521, merged via #525) decides whether an NVENC encoder (`h264_nvenc`/`hevc_nvenc`) is runnable by running a **1-frame test encode** and falling back to libx264 if it fails. The probe sourced its frame from `color=c=black:s=64x64:d=0.1`.

**Bug:** `64x64` is **below NVENC's minimum encode dimensions**, so on a real GPU the probe fails ?

> Nothing was written into output file, because at least one of its streams received no packets. Conversion failed!

? **even on boxes where NVENC works fine at real resolutions**.

**Impact:** on any GPU box whose NVENC minimum exceeds 64x64 ? **potentially including the L4 Cloud Run worker** ? `_select_encoder` silently falls back to libx264, so the 4K watermark re-encode stays on slow CPU and **#521's entire NVENC-on-L4 optimization is defeated**. It went unnoticed because the 2026-07-08 CUJ that motivated #521 measured the control-plane (libx264) path, not a live L4 NVENC stitch.

**Empirical characterization** (RTX 2070 Super, ffmpeg 8.1.1): `h264_nvenc` probe **FAILs at 64x64 and 128x128**, **PASSes at 145x49 / 256x256 / 320x240**, and a real 4K encode works. Same failures with a real `testsrc2` source ? so it's NVENC's dimension floor, not a `color`-source artifact.

## Fix

- Probe from a module constant `_PROBE_SOURCE_SIZE = "256x256"` (a safe margin above the observed
  floor) instead of `64x64`, with a comment documenting **why it must never shrink below the NVENC
  minimum**. It stays a real 1-frame encode to `null` (not an `-encoders` grep), so it still
  distinguishes compiled-in-but-unusable NVENC from genuinely usable.

- **Tests** (`tests/test_compiler.py`): the existing encoder tests all monkeypatch the whole probe
  away, so the real command geometry was never exercised. Adds:
  - `test_probe_encoder_uses_safe_resolution` ? parses the built ffmpeg argv and asserts `s=WxH` is
    **>= 256 on both axes**, guarding against regressing back to a sub-minimum size (it fails against
    the old `64x64`).
  - `test_probe_encoder_builds_single_frame_null_encode` ? probe stays a 1-frame null encode of the
    requested codec, not a capability grep.
  - `test_probe_encoder_returns_false_on_ffmpeg_failure` ? non-zero ffmpeg exit ? fall back.

Refs #521, #525.

## Checks

- [x] `python run_all_tests.py` green (**1878 passed, 8 skipped**); `ruff check .` clean;
  `mypy backend training` clean (mypy 2.1.0, the CI-pinned version). Coverage not lowered ? this PR
  only adds tests and exercises the previously-untested probe path.
- [x] Internal markdown links valid ? **N/A, no markdown files changed**.

## Owner verification (machine-side, per project convention)

- [ ] On a real NVENC box, confirm `h264_nvenc` now probes **PASS** at 256x256 (and that a live L4
  stitch actually uses `-c:v h264_nvenc`, not the libx264 fallback). I don't have GPU hardware to
  confirm the live encode.

#### 83. user (2026-07-08T06:32:57.067Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer/b55cb444-5f5e-4770-ad71-cee76683ff71/scratchpad/pr_body.md (file state is current in your context ? no need to Read it back)

#### 84. user (2026-07-08T06:33:17.071Z)

https://github.com/Khelsutra/badminton-highlight-indexer/pull/527

#### 85. assistant (2026-07-08T06:33:26.397Z)

PR opened: **https://github.com/Khelsutra/badminton-highlight-indexer/pull/527**. Let me confirm CI kicked off.

#### 86. user (2026-07-08T06:33:34.767Z)

PR #527  OPEN  fix/nvenc-probe-resolution -> master  mergeable=MERGEABLE
=== checks ===
Golden fixtures cross-OS (macos-latest) pending 0  https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28922638853/job/85803026033
Leak scan (PII / attribution guard)  pass  6s  https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28922638853/job/85803026022
Markdown link check (live docs) pass  11s  https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28922638853/job/85803026012
Ruff lint  pass  11s  https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28922638853/job/85803026020
Built wheel smoke (install + launch)  pending 0  https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28922638853/job/85803026075
Dependency license scan (no copyleft in runtime deps)  pending 0  https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28922638853/job/85803026025
Full test suite (offline, mocked)  pending 0  https://github.com/Khelsutra/badminton-

highlight-indexer/actions/runs/28922638853/job/85803026155
Golden fixtures cross-OS (ubuntu-latest)  pending 0
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28922638853/job/85803026010
Golden fixtures cross-OS (windows-latest)  pending 0
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28922638853/job/85803026028
Mypy (lenient baseline) pending 0  https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28922638853/job/85803026017
Syntax sweep + import smoke (no GPU stack)  pending 0
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28922638853/job/85803026030

**87. assistant (2026-07-08T06:34:00.724Z)**

CI is running ? the fast gates (**Ruff lint, Markdown link check, Leak scan**) already **pass**,
and the PR is `MERGEABLE` into `master`. The heavier jobs (full suite cross-OS, mypy, golden
fixtures) are pending. Since my local run already covered those, I'll let CI confirm in the
background rather than burn cycles polling.

**88. user (2026-07-08T06:34:02.696Z)**

Command running in background with ID: batzvc9df. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-khelsutra-guru-badminton-
highlight-indexer\b55cb444-5f5e-4770-ad71-cee76683ff71\tasks\batzvc9df.output. You will be
notified when it completes. To check interim output, use Read on that file path.

**89. assistant (2026-07-08T06:34:20.931Z)**

While CI runs, let me record this follow-up in memory (it's cross-session project state ? an
open PR blocked on owner GPU verification ? not something the repo already captures on its own).

**90. assistant (2026-07-08T06:34:33.284Z)**

---
name: pr-527-nvenc-probe-resolution
description: PR #527 fixes the NVENC availability probe (64x64?256x256) that silently defeated
#521's NVENC-on-L4 win; open, awaits owner live-GPU confirmation.
metadata:
  type: project
---

[PR #527](https://github.com/Khelsutra/badminton-highlight-indexer/pull/527) (branch
`fix/nvenc-probe-resolution`, off `origin/master`) fixes a bug in `_probe_encoder_available`
(`backend/pipeline/stitcher/compiler.py`, added by #521 / merged via #525, see [[pr-525-compile-
on-l4]]):
the 1-frame NVENC probe encoded from `color=c=black:s=64x64:d=0.1`, but **64x64 is below NVENC's
minimum encode dimensions**, so on a real GPU the probe FAILS even when NVENC works at real
resolutions ? `_select_encoder` silently falls back to libx264 ? #521's NVENC-on-L4 4K re-encode
win is defeated. Fix: bump to a `_PROBE_SOURCE_SIZE = "256x256"` constant.

**Why:** empirically (owner's RTX 2070 Super, ffmpeg 8.1.1) h264_nvenc probe FAILs at 64x64 &
128x128
but PASSes at ~145px+ (256/320 all pass, 4K works). Likely never caught because the 2026-07-08
CUJ
that motivated #521 measured the libx264 control-plane path, not a live L4 NVENC stitch.

**How to apply:** verified locally green ? full suite (1878 passed / 8 skipped), `ruff check .`,
`mypy backend training` (2.1.0). **Still owner-gated:** confirm on a real NVENC box that
h264_nvenc
now probes PASS at 256x256 AND that a live L4 stitch actually emits `-c:v h264_nvenc` (not the
libx264 fallback) ? I have no GPU to confirm the live encode.

**Repo hygiene note (2026-07-08):** did this in an isolated `git worktree`

(`../bhi-nvenc-probe-wt`) because the main checkout had conflicted sibling WIP ? `UU` marks on `orchestration.py` + `compiler.py`, a `sd-reel-engine-wip` stash, branch `codex/issue-523-sd-reel-plan`,
**no merge actually in progress**. That WIP was left untouched.

**91. user (2026-07-08T06:34:33.338Z)**

File created successfully at: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\memory\pr-527-nvenc-probe-resolution.md (file state is current in your context ? no need to Read it back)

**92. assistant (2026-07-08T06:34:37.763Z)**

Now the MEMORY.md index pointer. Let me read it and append the entry.

**93. user (2026-07-08T06:34:38.498Z)**

1 # Memory index
2
3 - [PR #525 ? compile-on-L4 (#521)](pr-525-compile-on-l4.md) ? config-driven phase?machine placement + L4 NVENC stitch; Part 3 (cheaper watermark) deferred.
4

**94. user (2026-07-08T06:34:43.584Z)**

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\memory\MEMORY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

**95. assistant (2026-07-08T06:34:53.173Z)**

Memory recorded. Let me confirm I left the main checkout's sibling WIP completely untouched (the whole point of the worktree).

**96. user (2026-07-08T06:34:57.881Z)**

```
=== main repo branch ===
codex/issue-523-sd-reel-plan
=== main repo status (should be the SAME sibling WIP as session start) ===
M  README.md
M  SESSION_HANDOFF.md
M  backend/config/models.py
M  backend/infrastructure/database_jobs.py
MM backend/main.py
UU backend/orchestration.py
UU backend/pipeline/stitcher/compiler.py
 M backend/worker/__main__.py
M  config.json
M  tests/test_api_endpoints.py
M  tests/test_async_jobs.py
 M tests/test_compile_dispatch.py
M  tests/test_compiler.py
M  tests/test_watermark.py
 M tests/test_worker.py
?? score_golden_gate.py
?? score_local.py
=== worktrees ===
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer
dac6736 [codex/issue-523-sd-reel-plan]
C:/kg-wt521
8866ffe [feat/521-compile-on-l4]
C:/kg-wt525fix
dac6736 [fix/521-worker-mirror-required]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
```

```
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/cfg-wt        20b2d5a [claude/wasb-
config-validation]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/compile-wt    b09a1d4 [claude/compile-
reliability]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/pr458-pre-wt  355dad6 (detached HEAD)
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/pr458-wt      5de1ef2 (detached HEAD)
C:/Users/avidu/AppData/Local/Temp/claude/wt524-969
bf97700 [feat/524-l4-primary]
C:/Users/avidu/Projects/khelsutra-guru/bhi-nvenc-probe-wt
7d6c049 [fix/nvenc-probe-resolution]
C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-cheap-watermark
dac6736 [perf/521-cheap-watermark]
```

**97. assistant (2026-07-08T06:35:25.915Z)**

Confirmed: the main checkout is untouched by me ? still on `codex/issue-523-sd-reel-plan` with its sibling WIP. In fact it now shows *more* modified files than at session start (`MM backend/main.py`, `tests/test_worker.py`, ?), so a **sibling session is actively editing that checkout right now** ? which fully validates doing this in an isolated worktree. There's a whole fleet of worktrees here; mine (`bhi-nvenc-probe-wt` @ `7d6c049`) sits cleanly alongside them.

**PR is up: [#527](https://github.com/Khelsutra/badminton-highlight-indexer/pull/527)** (`fix/nvenc-probe-resolution` ? `master`, mergeable). CI is watching in the background; I'll confirm the full rollup and clean up the worktree once it settles.

Here's the summary while CI runs:

## What shipped

**Root cause** ? [`_probe_encoder_available`](backend/pipeline/stitcher/compiler.py) probed NVENC with a `64x64` test frame, which is *below NVENC's minimum encode size*, so the probe fails on real GPUs even when NVENC works at real resolutions ? `_select_encoder` silently falls back to libx264 ? #521's NVENC-on-L4 4K re-encode win is quietly defeated (never caught because the 2026-07-08 CUJ measured the libx264 control-plane path).

**Fix** ? probe from a documented `_PROBE_SOURCE_SIZE = "256x256"` constant (safe margin above the observed ~145px floor), still a real 1-frame null encode.

**Tests** ? added `test_probe_encoder_uses_safe_resolution` (parses the built argv, asserts `s=WxH` ? 256 on both axes ? the regression guard; it fails against the old `64x64`) plus two companions. The prior encoder tests all monkeypatched the probe away, so the real command geometry was never exercised.

## Verified locally (all green)

| Gate | Result |
|---|---|
| `python run_all_tests.py` | **1878 passed, 8 skipped** |
| `ruff check .` | clean |
| `mypy backend training` | clean (mypy 2.1.0 ? CI's pin) |
| Guard behavior | fails at `64x64`, passes at `256x256` |

## The one thing I can't verify ? needs you

On a real NVENC box, confirm `h264_nvenc` now probes **PASS** at 256x256 **and** that a live L4 stitch actually emits `-c:v h264_nvenc` (not the libx264 fallback). I have no GPU to confirm the live encode ? flagged as an owner check in the PR.

I'll report back when CI finishes.

**98. user (2026-07-08T06:36:09.685Z)**

<task-notification>

<task-id>batzvc9df</task-id>
<tool-use-id>toolu_01QSTcigw6BfQvizUXj8S8BY</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\b55cb444-5f5e-4770-ad71-cee76683ff71\tasks\batzvc9df.output</output-file>
<status>completed</status>
<summary>Background command "Watch CI checks to completion (background)" completed (exit code 0)</summary>
</task-notification>

**99. assistant (2026-07-08T06:36:13.184Z)**

CI watch completed with **exit code 0** (all checks passed). Let me confirm the final rollup.

**100. user (2026-07-08T06:36:22.977Z)**

```
Dependency license scan (no copyleft in runtime deps)   pass    33s
https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28922638853/job/85803026025
Golden fixtures cross-OS (ubuntu-latest)        pass    28s
https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28922638853/job/85803026010
Golden fixtures cross-OS (windows-latest)       pass    1m2s
https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28922638853/job/85803026028
Built wheel smoke (install + launch)    pass    1m19s   https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28922638853/job/85803026075
Dependency license scan (no copyleft in runtime deps)   pass    33s
https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28922638853/job/85803026025
Full test suite (offline, mocked)       pass    2m37s   https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28922638853/job/85803026155
Golden fixtures cross-OS (macos-latest) pass    25s     https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28922638853/job/85803026033
Golden fixtures cross-OS (ubuntu-latest)        pass    28s
https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28922638853/job/85803026010
Golden fixtures cross-OS (windows-latest)       pass    1m2s
https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28922638853/job/85803026028
Leak scan (PII / attribution guard)     pass    6s      https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28922638853/job/85803026022
Markdown link check (live docs) pass    11s     https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28922638853/job/85803026012
Mypy (lenient baseline) pass    49s     https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28922638853/job/85803026017
Ruff lint       pass    11s     https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28922638853/job/85803026020
Syntax sweep + import smoke (no GPU stack)      pass    24s
https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28922638853/job/85803026030
Built wheel smoke (install + launch)    pass    1m19s   https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28922638853/job/85803026075
Dependency license scan (no copyleft in runtime deps)   pass    33s
https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28922638853/job/85803026025
Full test suite (offline, mocked)       pass    2m37s   https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28922638853/job/85803026155
Golden fixtures cross-OS (macos-latest) pass    25s     https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28922638853/job/85803026033
Golden fixtures cross-OS (ubuntu-latest)        pass    28s
https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28922638853/job/85803026010
Golden fixtures cross-OS (windows-latest)       pass    1m2s
https://github.com/Khelsutra/badminton-highlight-
```

```
indexer/actions/runs/28922638853/job/85803026028
Leak scan (PII / attribution guard)     pass    6s      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28922638853/job/85803026022
Markdown link check (live docs) pass    11s     https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28922638853/job/85803026012
Mypy (lenient baseline) pass    49s     https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28922638853/job/85803026017
Ruff lint       pass    11s     https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28922638853/job/85803026020
Syntax sweep + import smoke (no GPU stack)      pass    24s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28922638853/job/85803026030
=== final rollup exit: 0 ===
```

**101. assistant (2026-07-08T06:36:34.775Z)**

**All CI checks green** (exit 0): full suite (offline, mocked), mypy, ruff, golden fixtures on ubuntu/windows/macos, wheel smoke, license scan, leak scan, link check, syntax/import smoke. Now let me clean up my worktree ? the branch and PR live on the remote, so it's safe to remove.

**102. user (2026-07-08T06:36:38.564Z)**

```
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/bhi-nvenc-probe-wt': Permission
denied
error: failed to delete '.git/worktrees/bhi-nvenc-probe-wt': Permission denied
(nvenc-probe worktree gone ? branch still on origin)
```

**103. assistant (2026-07-08T06:37:00.374Z)**

The removal hit a Windows file lock (`Permission denied`) ? likely a lingering handle from the background CI-watch that had its cwd in that dir. This is just cleanup (work is committed, pushed, CI-green ? a leftover worktree is harmless). Let me check the true state and retry.

**104. user (2026-07-08T06:37:03.686Z)**

```
pwd: /c/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer
=== worktree list ===
(not registered)
=== dir still present? ===
DIR EXISTS
```

**105. assistant (2026-07-08T06:37:17.627Z)**

The worktree is already **unregistered** from git ? only an orphaned directory remains on disk (work is safely committed + pushed). Let me delete the leftover directory.

**106. user (2026-07-08T06:37:20.144Z)**

```
dir removed cleanly
```

**107. assistant (2026-07-08T06:37:27.075Z)**

Clean. Let me do a final state check ? prune any stale admin entry and confirm the branch ref survived.

**108. user (2026-07-08T06:37:35.544Z)**

```
error: failed to delete '.git/worktrees/a1-master-worktree': Permission denied
error: failed to delete '.git/worktrees/bhi-nvenc-probe-wt': Permission denied
error: failed to delete '.git/worktrees/bhi-wt-worker-reval': Permission denied
error: failed to delete '.git/worktrees/fix-worktree': Permission denied
error: failed to delete '.git/worktrees/master-compare': Permission denied
error: failed to delete '.git/worktrees/pr465': Permission denied
```

```
error: failed to delete '.git/worktrees/pr465-current-merge': Permission denied
error: failed to delete '.git/worktrees/pr465-merge': Permission denied
error: failed to delete '.git/worktrees/pr467': Permission denied
error: failed to delete '.git/worktrees/pr469': Permission denied
error: failed to delete '.git/worktrees/wt-465fix': Permission denied
error: failed to delete '.git/worktrees/wt-a1-runlog': Permission denied
error: failed to delete '.git/worktrees/wt-claudemd-r25': Permission denied
error: failed to delete '.git/worktrees/wt-cloudrun-cancel': Permission denied
error: failed to delete '.git/worktrees/wt-cloudrun-dispatch': Permission denied
error: failed to delete '.git/worktrees/wt-cp-rebuild-spec': Permission denied
error: failed to delete '.git/worktrees/wt-dockerfile-auth': Permission denied
error: failed to delete '.git/worktrees/wt-docs': Permission denied
error: failed to delete '.git/worktrees/wt-docs-reconcile': Permission denied
error: failed to delete '.git/worktrees/wt-drain-concurrency': Permission denied
error: failed to delete '.git/worktrees/wt-ffmpeg-timeouts': Permission denied
error: failed to delete '.git/worktrees/wt-gdrive-api-safety': Permission denied
error: failed to delete '.git/worktrees/wt-idempotent-process': Permission denied
error: failed to delete '.git/worktrees/wt-input-jail': Permission denied
error: failed to delete '.git/worktrees/wt-output-dir-config': Permission denied
error: failed to delete '.git/worktrees/wt-p1-genyaml': Permission denied
error: failed to delete '.git/worktrees/wt-p2-upload-stream': Permission denied
error: failed to delete '.git/worktrees/wt-p4-md5cache': Permission denied
error: failed to delete '.git/worktrees/wt-p5a-pollknob': Permission denied
error: failed to delete '.git/worktrees/wt-p6-heartbeat': Permission denied
error: failed to delete '.git/worktrees/wt-pr481-review': Permission denied
error: failed to delete '.git/worktrees/wt-retention-sweep': Permission denied
error: failed to delete '.git/worktrees/wt-review-492-mergecheck': Permission denied
error: failed to delete '.git/worktrees/wt-review-493-mergecheck': Permission denied
error: failed to delete '.git/worktrees/wt-scratch-reclaim': Permission denied
error: failed to delete '.git/worktrees/wt-signed-download': Permission denied
error: failed to delete '.git/worktrees/wt-source-cache': Permission denied
error: failed to delete '.git/worktrees/wt-upload-dir': Permission denied
error: failed to delete '.git/worktrees/wt-upload-staging': Permission denied
=== my branch ref intact? ===
7d6c049f270116fecd1cf4b6976150a02d57b1e9
=== branch tracks origin ===
fix/nvenc-probe-resolution -> origin/fix/nvenc-probe-resolution
=== main checkout branch unchanged ===
codex/issue-523-sd-reel-plan
```

**109. assistant (2026-07-08T06:38:23.098Z)**

Everything is in order. That `git worktree prune` permission error is an **environment-wide condition** ? it failed identically on all **38** pre-existing worktree admin stubs in this repo (a heavily parallelized multi-session checkout with a `.git` lock, not something I caused). What matters is verified clean:

- **My branch ref intact** ? `fix/nvenc-probe-resolution` ? `7d6c049`, tracking `origin/fix/nvenc-probe-resolution`.
- **`git worktree list` no longer shows my worktree**; the directory is deleted. Only a benign, un-prunable `.git/worktrees/bhi-nvenc-probe-wt` admin stub remains (matching the 38 others) ? inert, ignored by git.
- **Main checkout untouched** ? still on `codex/issue-523-sd-reel-plan` with the sibling's WIP preserved.

---

## Done ?

**[PR #527: fix(serving): probe NVENC at 256x256, not 64x64 (#521/#525)](https://github.com/Khelsutra/badminton-highlight-indexer/pull/527)** ? open, mergeable into `master`, **all CI green**.

**The fix** ([backend/pipeline/stitcher/compiler.py](backend/pipeline/stitcher/compiler.py)):

`_probe_encoder_available` now encodes its 1-frame probe from a documented `_PROBE_SOURCE_SIZE = "256x256"` constant instead of the sub-minimum `64x64`. It stays a real null encode ? I only changed the source geometry ? so it still distinguishes compiled-in-but-unusable NVENC from genuinely usable. The constant carries a comment with your RTX 2070S / ffmpeg 8.1.1 characterization so it can't be "optimized" back down.

**The tests** ([tests/test_compiler.py](tests/test_compiler.py)): the existing encoder tests all monkeypatched the whole probe away, so its real command geometry was never exercised. Added `test_probe_encoder_uses_safe_resolution` (parses the built ffmpeg argv, asserts `s=WxH` ? 256 on both axes ? the regression guard; provably fails against `64x64`, passes at `256x256`), plus companions for the single-frame-null shape and the fall-back-on-nonzero-exit path.

**Verification** ? local and CI both fully green:

| Gate | Local | CI |
|---|---|---|
| Full suite (`run_all_tests.py`) | 1878 passed, 8 skipped | ? pass |
| `ruff check .` | clean | ? pass |
| `mypy backend training` (2.1.0) | clean | ? pass |
| Golden fixtures ubuntu/windows/macos | ? | ? pass |
| wheel smoke · license scan · leak scan · link check · import smoke | ? | ? pass |

Coverage isn't lowered ? the PR only adds tests and newly exercises the previously-untested probe path.

## Still needs you (hardware I don't have) ?

On a real NVENC box: confirm `h264_nvenc` now probes **PASS** at 256x256 **and** that a live L4 stitch actually emits `-c:v h264_nvenc` rather than the libx264 fallback. This is the real-world proof that #521's NVENC-on-L4 win is restored ? it's flagged as an owner check in the PR. (Recorded in memory as [pr-527-nvenc-probe-resolution](file://C:/Users/avidu/.claude/projects/C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer/memory/pr-527-nvenc-probe-resolution.md) so it survives across sessions.)

One thing worth flagging beyond the task: this bug means #521's L4 optimization has **likely been silently inert in production** since it merged ? every L4 stitch quietly falling back to CPU libx264 ? so it's worth prioritizing that live check.

### 110. user (2026-07-08T06:40:36.744Z)

can you file a bug to send an alert everytime the GPU stitch silently falls back to CP? this will help us set up some monitoring as well

### 111. assistant (2026-07-08T06:41:05.488Z)

Good follow-up ? the root lesson of #527 is exactly that a silent fallback with only a `logger.warning` went unnoticed for weeks. Before I file, let me ground the issue in the repo's actual observability conventions so it proposes something that fits (not an invented mechanism). Let me map what alerting/metrics primitives already exist.

### 112. user (2026-07-08T06:41:07.200Z)

Found 11 files
backend\orchestration.py
backend\eval\fusion_audio.py
backend\eval\fusion_golden.py
backend\utils\run_telemetry.py
backend\reporting\run_signals.py
backend\reporting__init__.py
backend\reporting\harvest.py
backend\eval\per_user_regression.py
backend\eval\per_user_gate.py
backend\eval\golden_real_fixtures.py
backend\reporting\spec.yaml

**113. user (2026-07-08T06:41:09.210Z)**

```
<persisted-output>
Output too large (21.2KB). Full output saved to: C:\Users\avidu\.claude\projects\C--Users-avidu-
Projects-khelsutra-guru-badminton-highlight-indexer\b55cb444-5f5e-4770-ad71-cee76683ff71\tool-
results\toolu_01Wbso6yhpZpoBKfNcU3WRnp.txt

Preview (first 2KB):
backend\annotations\base.py-96-        except Exception:
backend\annotations\base.py:97:            # Providers needing constructor args fall back to the
class name.
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\annotations\base.py-98-            key = getattr(provider_cls, "__name__",
"unknown")
--
backend\config\loader.py-68-            "[CONFIG] unrecognized DEPLOYMENT_PROFILE env value %r
(valid: %s); ignoring it "
backend\config\loader.py:69:            "and falling back to config.json deployment.profile.",
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\config\loader.py-70-
ep,
--
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\config\models.py-391-
# dominated the 2026-07-08 CUJ stitch (#521). The compiler probes the requested encoder ONCE and
backend\config\models.py:392:    # falls back to libx264 if this ffmpeg build can't run it, so a
non-GPU box never breaks. The
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\config\models.py-393-
# cloud compile-dispatch forwards ``deployment.cloud_stitch_encoder`` onto this for the L4 run;
--
backend\config\models.py-453-    backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"]
= "local"
backend\config\models.py:454:    # Output root for produced artifacts. "" => fall back to
output.output_dir (non-breaking).
backend\config\models.py-455-    output_root: str = ""
--
backend\server_runtime.py-82-def _resolve_app_port(host: str, port: int) -> int:
backend\server_runtime.py:83:    """For --app: keep `port` if free, else fall back to a free one
so a busy 8000 never blocks the app."""
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\server_runtime.py-84-
return port if _port_is_free(host, port) else _find_free_port(host)
--
backend\orchestration.py-388-                # Read-path re-hydrate (#485): a wiped DB has no
source record. The report
...
</persisted-output>
```

**114. user (2026-07-08T06:41:09.671Z)**

```
backend\eval\metrics.py
backend\flywheel.py
```

**115. assistant (2026-07-08T06:41:16.395Z)**

There's a real telemetry/reporting surface here (`run_telemetry.py`, `reporting/run_signals.py`). Let me read those to understand the existing signal mechanism the issue should hook into.

**116. user (2026-07-08T06:41:16.546Z)**

```
1       """Structured run telemetry ? a compact, rotating JSONL "black box" for long/flaky runs.
2
3       A long GPU/CV run (the fusion-golden extraction, a WSL/WASB pass, an ffmpeg stitch) can
die with
4       **no Python traceback** ? the OS kills it (a console-control event, an OOM, a thermal
event). When
```

```
    that happens the only forensic evidence is whatever was last flushed to disk. This
module keeps a
    tiny (~10 KB) rotating runlog of *execution velocity + machine health* so a post-mortem
is a
    bullseye instead of a guess: "it was at 26 fps, GPU 85 °C throttled to 1125/2100 MHz,
4.8 GB VRAM
    free, CPU 60 %, RAM 21/32 GB ? then vanished."

    It is the **observability** companion to the `ExecutionPolicy` *control* layer
    (`docs/EXECUTION_POLICY.md`): the policy decides wait / abort / resume; this records the
signals
    that explain *why* a run slowed or died, and lets us compare execution velocity across
quality
    iterations.

    Design:
    - **Caller-driven snapshots** (no background thread): call ``snapshot(done, total)``
from an
      existing progress callback; fps is derived from the delta since the previous snapshot.
      ``event(kind, msg)`` records discrete moments (start, video-done, error) WITH a health
sample, so
      the last line before a kill captures the machine state at death.
    - **Capped + rotating**: a fixed ``meta`` header line (machine / GPU / start) is always
kept; recent
      snapshot/event lines form an on-disk ring trimmed to ``max_bytes`` (~10 KB). The file
is rewritten
      atomically (tmp + ``os.replace``) on each write, so a hard kill never leaves it half-
written.
    - **Graceful degradation**: GPU stats come from ``nvidia-smi`` (absent -> ``None``);
system stats
      from ``psutil`` (not installed -> ``None``). Collection never raises into the caller ?
a telemetry
      hiccup must not break the run it observes.
    - **Deterministic / testable**: the clock and the two collectors are injectable, so fps,
rotation,
      and parsing are unit-tested with no real GPU, no psutil, and no wall-clock.

    All stdlib + optional ``psutil`` (BSD-3, commercial-clean). No network, no training-data
path.
    """

    from __future__ import annotations

    import json
    import os
    import shutil
    import subprocess
    import time
    from typing import Any, Callable, Dict, List, Optional

    GpuFn = Callable[[], Optional[Dict[str, Any]]]
    SysFn = Callable[[], Optional[Dict[str, Any]]]

    # nvidia-smi fields pulled in one query, in this order (see gpu_stats parsing).
    _GPU_QUERY =
"temperature.gpu,utilization.gpu,power.draw,clocks.sm,clocks.max.sm,memory.used,memory.free"
    _THROTTLE_CLOCK_FRAC = 0.8  # SM clock below this fraction of max => flagged throttled
    _THROTTLE_TEMP_C = (
        84.0  # ... or temperature at/above this (laptop Max-Q throttle territory)
    )


    def _num(s: str) -> Optional[float]:
        try:
            return float(s)
```

```
55          except (TypeError, ValueError):
56              return None
57
58
59      def gpu_stats(timeout: float = 4.0) -> Optional[Dict[str, Any]]:
60          """One ``nvidia-smi`` sample as a dict, or ``None`` if nvidia-smi is unavailable /
errors.
61
62          Returns util %, temperature, power (W), current + max SM clock (MHz), VRAM used /
free (MB), and a
63          derived ``throttled`` flag (SM clock well below max, or hot). Pure read; never
raises.
64          """
65          exe = shutil.which("nvidia-smi")
66          if not exe:
67              return None
68          try:
69              out = subprocess.run(
70                  [exe, f"--query-gpu={_GPU_QUERY}", "--format=csv,noheader,nounits"],
71                  capture_output=True,
72                  text=True,
73                  timeout=timeout,
74                  check=True,
75              ).stdout.strip()
76          except (subprocess.SubprocessError, OSError):
77              return None
78          row = out.splitlines()[0] if out else ""
79          parts = [p.strip() for p in row.split(",")]
80          if len(parts) < 7:
81              return None
82          vals = [_num(p) for p in parts[:7]]
83          temp, util, power, sm, sm_max, used, free = vals
84          throttled = bool(temp is not None and temp >= _THROTTLE_TEMP_C)
85          if sm is not None and sm_max:
86              throttled = throttled or sm < _THROTTLE_CLOCK_FRAC * sm_max
87          return {
88              "util_pct": util,
89              "temp_c": temp,
90              "power_w": power,
91              "sm_mhz": sm,
92              "sm_max_mhz": sm_max,
93              "vram_used_mb": used,
94              "vram_free_mb": free,
95              "throttled": throttled,
96          }
97
98
99      def system_stats() -> Optional[Dict[str, Any]]:
100         """CPU % + RAM via ``psutil``, or ``None`` if psutil isn't installed. Never
raises."""
101         try:
102             import psutil  # optional dep (BSD-3); only used when present
103         except ImportError:
104             return None
105         try:
106             vm = psutil.virtual_memory()
107             return {
108                 "cpu_pct": psutil.cpu_percent(interval=None),
109                 "ram_used_gb": round(vm.used / 1e9, 2),
110                 "ram_total_gb": round(vm.total / 1e9, 2),
111                 "ram_pct": vm.percent,
112             }
113         except Exception:  # noqa: BLE001 - psutil edge cases must not break the run
114             return None
115
```

```
116
117     def gpu_name() -> Optional[str]:
118         """The GPU model string (for the runlog header), or ``None`` without nvidia-smi."""
119         exe = shutil.which("nvidia-smi")
120         if not exe:
121             return None
122         try:
123             out = subprocess.run(
124                 [exe, "--query-gpu=name", "--format=csv,noheader"],
125                 capture_output=True,
126                 text=True,
127                 timeout=4.0,
128                 check=True,
129             ).stdout
130         except (subprocess.SubprocessError, OSError):
131             return None
132         return out.splitlines()[0].strip() if out else None
133
134
135     class RunTelemetry:
136         """A small rotating JSONL runlog of execution velocity + machine health.
137
138         Call ``snapshot(done, total, phase=...)`` from a progress callback and ``event(kind,
msg)`` at
139         milestones. The file holds a ``meta`` header line plus the most recent snapshot /
event lines
140         that fit within ``max_bytes`` (~10 KB), rewritten atomically on every write.
141         """
142
143         def __init__(
144             self,
145             path: str,
146             label: str = "run",
147             max_bytes: int = 10_000,
148             gpu_fn: GpuFn = gpu_stats,
149             sys_fn: SysFn = system_stats,
150             clock: Callable[[], float] = time.time,
151         ) -> None:
152             self.path = path
153             self.label = label
154             self.max_bytes = max_bytes
155             self._gpu_fn = gpu_fn
156             self._sys_fn = sys_fn
157             self._clock = clock
158             self._t0 = clock()
159             self._last_done: Optional[int] = None
160             self._last_t: Optional[float] = None
161             self._lines: List[
162                 str
163             ] = []  # recent body lines (on-disk ring, trimmed to max_bytes)
164             self._meta = json.dumps(
165                 {
166                     "meta": {
167                         "label": label,
168                         "started": self._iso(self._t0),
169                         "pid": os.getpid(),
170                         "gpu_name": gpu_name(),
171                         "cpu_count": os.cpu_count(),
172                     }
173                 },
174                 separators=(",", ":"),
175             )
176             self._write()
177
178         # -- public API ---------------------------------------------------------
```

```
        def snapshot(
            self,
            done: Optional[int] = None,
            total: Optional[int] = None,
            phase: Optional[str] = None,
            **extra: Any,
        ) -> Dict[str, Any]:
            """Record a velocity + health sample; ``fps`` is derived from the previous
snapshot."""
            now = self._clock()
            rec: Dict[str, Any] = {
                "t": self._iso(now),
                "elapsed_s": round(now - self._t0, 1),
            }
            if phase is not None:
                rec["phase"] = phase
            if done is not None:
                rec["done"] = done
                if total:
                    rec["total"] = total
                    rec["pct"] = round(100.0 * done / total, 1)
                if (
                    self._last_done is not None
                    and self._last_t is not None
                    and now > self._last_t
                ):
                    rec["fps"] = round((done - self._last_done) / (now - self._last_t), 1)
                self._last_done, self._last_t = done, now
            rec.update(extra)
            self._attach_health(rec)
            self._append(rec)
            return rec

        def event(self, kind: str, msg: str = "", **extra: Any) -> Dict[str, Any]:
            """Record a milestone (start, video_done, error, restart) WITH a health
sample."""
            now = self._clock()
            rec: Dict[str, Any] = {
                "t": self._iso(now),
                "elapsed_s": round(now - self._t0, 1),
                "event": kind,
            }
            if msg:
                rec["msg"] = msg
            rec.update(extra)
            self._attach_health(rec)
            self._append(rec)
            return rec

        # -- internals ------------------------------------------------------------
        def _attach_health(self, rec: Dict[str, Any]) -> None:
            gpu = self._safe(self._gpu_fn)
            sys_ = self._safe(self._sys_fn)
            if gpu is not None:
                rec["gpu"] = gpu
            if sys_ is not None:
                rec["sys"] = sys_

        def _append(self, rec: Dict[str, Any]) -> None:
            self._lines.append(json.dumps(rec, separators=(",", ":")))
            budget = self.max_bytes - len(self._meta) - 1
            while len(self._lines) > 1 and sum(len(s) + 1 for s in self._lines) > budget:
                self._lines.pop(0)  # drop oldest body line; the meta header is always kept
            self._write()

```

```
        def _write(self) -> None:
            try:
                tmp = self.path + ".tmp"
                with open(tmp, "w", encoding="utf-8") as f:
                    f.write(self._meta + "\n")
                    for s in self._lines:
                        f.write(s + "\n")
                os.replace(tmp, self.path)
            except OSError:
                pass  # telemetry must never break the run it observes

        @staticmethod
        def _safe(fn: GpuFn) -> Optional[Dict[str, Any]]:
            try:
                return fn()
            except Exception:  # noqa: BLE001 - a collector hiccup must not break the run
                return None

        @staticmethod
        def _iso(t: float) -> str:
            return time.strftime("%Y-%m-%dT%H:%M:%S", time.localtime(t))

```

**117. assistant (2026-07-08T06:41:22.905Z)**

`RunTelemetry` is the local forensic black-box with an `event()` API. Now the reporting/signals surface ? that's likely the monitoring-facing layer. Let me read it.

**118. user (2026-07-08T06:41:23.883Z)**

```
      """In-process signal channel: pipeline internals -> the per-video report.

      Some report-worthy facts (e.g. the provider refusing an oversize chunk) surface
      deep inside a segmenter's worker threads, with no return path to the harvest ?
      segmenters return only their segment list. Layers record those facts here keyed
      by video_id; `emit_indexer_report` pops them when the run's report is built.

      Process-local and best-effort BY DESIGN: recording must never break a run, the
      store is consumed (popped) once per report, and signals for a video are simply
      dropped if no report is emitted. No obsreport dependency ? importable everywhere.
      """

      from __future__ import annotations

      import threading
      from typing import Any, Dict

      _lock = threading.Lock()
      _signals: Dict[str, Dict[str, Any]] = {}


      def incr(video_id: str, key: str, by: int = 1) -> None:
          """Add `by` to the counter `key` for this video's in-flight run."""
          if not video_id or by == 0:
              return
          with _lock:
              sig = _signals.setdefault(video_id, {})
              sig[key] = int(sig.get(key, 0)) + by


      def pop(video_id: str) -> Dict[str, Any]:
          """Consume (remove and return) all signals recorded for this video's run."""
          with _lock:
              return _signals.pop(video_id, {})

```

**119. assistant (2026-07-08T06:41:39.578Z)**

Now I understand the two surfaces: `RunTelemetry.event()` (forensic runlog) and `run_signals.incr(video_id, key)` (pipeline?per-video report channel). Let me see how signals currently flow into a report and how `incr` is used elsewhere, so my proposal mirrors the real convention.

**120. user (2026-07-08T06:41:41.762Z)**

```
backend\main.py:70:from backend.reporting import emit_indexer_report
backend\main.py:1570:                paths = emit_indexer_report(
backend\worker\__main__.py:40:from backend.reporting import emit_indexer_report
backend\worker\__main__.py:381:        emit_indexer_report(
backend\reporting\__init__.py:5:`emit_indexer_report(...)` is designed to be called from a
`finally` block and
backend\reporting\__init__.py:13:from . import run_signals  # no obsreport dependency ? always
importable
backend\reporting\__init__.py:29:def emit_indexer_report(
backend\reporting\__init__.py:54:    signals = run_signals.pop(video_id)
backend\reporting\run_signals.py:6:by video_id; `emit_indexer_report` pops them when the run's
report is built.
backend\orchestration.py:16:``ProcessRequest``, ``compute_video_id``, ``emit_indexer_report``,
``VideoCompiler``.
backend\orchestration.py:1046:        paths = _main.emit_indexer_report(
backend\eval\fusion_golden.py:384:            telem.event("video_start", spec["name"])
backend\eval\fusion_golden.py:397:                telem.event("error", spec["name"], err=str(e))
backend\eval\fusion_golden.py:403:            telem.event(
backend\pipeline\segmenters\gemini.py:10:from backend.reporting import run_signals
backend\pipeline\segmenters\gemini.py:342:            run_signals.incr(video_id,
"oversize_clips_skipped", oversize_skips)
backend\pipeline\segmenters\gemini.py:358:            run_signals.incr(video_id,
"generate_failed_chunks", generate_fails)
backend\pipeline\segmenters\gemini.py:425:            run_signals.incr(video_id,
"oversize_clips_skipped", 1)
backend\pipeline\detectors\base.py:59:segmenter Results (or pass detector_health into
emit_indexer_report context), the
```

**121. user (2026-07-08T06:41:44.446Z)**

```
88:        # (_select_encoder) and falls back to libx264 if this ffmpeg build can't run it.
277:    def _select_encoder(self) -> Tuple[List[str], str]:
311:        video_codec_args: Optional[List[str]] = None,
377:            video_codec_args
378:            if video_codec_args is not None
526:            video_codec_args, chosen_encoder = self._select_encoder()
527:            if chosen_encoder != "libx264":
528:                logger.info("[compile] encoding clips with %s (hardware).", chosen_encoder)
539:                    video_codec_args,
```

**122. assistant (2026-07-08T06:42:10.633Z)**

Now I have the grounded convention: `run_signals.incr(video_id, key)` is exactly how report-worthy facts that surface deep in the pipeline (e.g. `oversize_clips_skipped`, `generate_failed_chunks`) get folded into the per-video report via `emit_indexer_report` (called in `finally` in `main.py`/`worker`/`orchestration`). Before filing, let me check for an existing duplicate issue and the available labels.

**123. user (2026-07-08T06:42:15.958Z)**

```
=== existing open issues (encoder/nvenc/fallback/monitoring) ===
331     OPEN    [perf] Hosted processing is CPU-bound (ffmpeg/cv2) ? measured per-phase
breakdown + levers (GPU/NVENC, parallel extraction, async compile)
2026-06-22T14:14:15Z
501     OPEN    [devex/infra] Checked-in canonical GPU VM setup script ? the default bring-up
```

```
template (GCP)            2026-07-06T17:39:07Z
483     OPEN    [cost] Downsample 'detectably long' uploads to a fast proxy for DETECTION (GPU
ffmpeg); build the reel from the ORIGINAL                2026-07-06T18:43:52Z
523     OPEN    Bug: optional SD reel mode should replace, not duplicate, original-resolution
stitching bug     2026-07-08T05:56:27Z
349     OPEN    WASB native: decode is CPU-bound & single-threaded in streaming mode ? L4 GPU
~80% idle (no NVDEC/parallel decode)              2026-07-06T19:56:44Z
524     OPEN    epic: collapse the pipeline onto the L4 worker; shrink control-plane to a thin
router (upload?stitch on L4 + async bucket copy) enhancement     2026-07-08T05:56:22Z
326     OPEN    Hosted-app observability: request tracing + replay for debugging production CUJ
failures                2026-06-22T13:54:40Z
526     OPEN    perf/latency: speculatively spin up the L4 worker at upload-start (warm-and-
wait) to hide cold-start    bug, enhancement        2026-07-08T06:03:25Z
=== labels ===
bug     Something isn't working #d73a4a
documentation   Improvements or additions to documentation      #0075ca
duplicate       This issue or pull request already exists       #cfd3d7
enhancement     New feature or request  #a2eeef
good first issue        Good for newcomers      #7057ff
help wanted     Extra attention is needed       #008672
invalid This doesn't seem right #e4e669
question        Further information is requested        #d876e3
wontfix This will not be worked on      #ffffff
D2              #ededed
Phase-1         #ededed
refactoring             #ededed
```

**124. assistant (2026-07-08T06:42:46.721Z)**

No duplicate ? closest neighbors are #331 (CPU-bound perf levers) and #326 (observability/tracing), both different scope. I'll reference them. Available labels include `bug`. Let me confirm one wiring detail ? whether `compile_highlights` has `video_id` in scope (determines the cleanest emission point) ? then write the issue.

**125. user (2026-07-08T06:42:49.702Z)**

```
458:    def compile_highlights(
459-        self,
460-        raw_video_path: str,
461-        segments: List[Tuple[float, float]],
462-        output_filename: str,
463-    ) -> str:
464-        """
465-        Extracts selected segments from a raw video and stitches them together
466-        into a seamless highlights file, applying end_padding to prevent abrupt endings.
```

**126. assistant (2026-07-08T06:42:56.259Z)**

Confirmed ? `compile_highlights` has no `video_id` in scope, so the issue should note both wiring options. Let me write the issue body.

**127. assistant (2026-07-08T06:43:27.490Z)**

## Problem

When the compiler is configured for a hardware encoder (`h264_nvenc` / `hevc_nvenc`) but the per-run probe reports it unrunnable, `_select_encoder` **silently falls back to libx264 (CPU)** and
the only trace is a single `logger.warning`. There is **no metric, no structured signal, and no alert** ? so a persistent fallback on a box that is *supposed* to have hardware encode is invisible
in production.

This is not hypothetical: [#527](https://github.com/Khelsutra/badminton-highlight-

indexer/pull/527)
fixed a probe bug (`64x64` test frame below NVENC's minimum encode size) that made **every**
NVENC
probe fail on real GPUs. Because the fallback was silent,
[#521](https://github.com/Khelsutra/badminton-highlight-indexer/pull/521)'s
"run the 4K watermark re-encode on the L4's NVENC" optimization was very likely **inert in
production from the day it merged** ? every L4 stitch quietly re-encoding on slow CPU libx264 ?
and
nobody could have known without reading logs on a live box. A monitoring signal would have
caught it
in the first run.

## Current behavior

`backend/pipeline/stitcher/compiler.py :: _select_encoder`:

```python
if enc in ("h264_nvenc", "hevc_nvenc") and not _probe_encoder_available(enc):
    logger.warning(
        "[compile] encoder %r is not runnable in this ffmpeg build ? falling back to "
        "libx264 (CPU). On the L4 worker this should be available.", enc,
    )
    enc = "libx264"
```

A `logger.warning` is not a monitorable signal. It is not harvested into the per-video report,
not
counted, and not alertable.

## Desired behavior

Emit a **structured signal every time a hardware?CPU encoder fallback happens**, so it (a) lands
in
the per-video report like our other pipeline signals and (b) can drive an **alert / monitor**.

**Alert condition (the important part ? keep it low-noise):** fire only when the deployment is
*configured for* NVENC (`deployment.cloud_stitch_encoder` / `stitching.encoder` ?
{`h264_nvenc`, `hevc_nvenc`}) but the run **chose libx264**. A box legitimately configured for
CPU
(`encoder = "libx264"`) never probes, never falls back, and must produce **no** signal and
**no**
alert. So the signal means exactly: *"a machine that should have hardware encode is silently on
CPU"* ? misconfig, missing/broken driver or ffmpeg build, or a #527-class regression.

## Proposed approach (grounded in existing conventions)

Mirror the established "report-worthy fact that surfaces deep in the pipeline" pattern ? the
same one
`gemini.py` uses for `oversize_clips_skipped` / `generate_failed_chunks`:

1. **Per-video report signal** ? on fallback, `run_signals.incr(video_id,
"gpu_encoder_fallback_to_cpu")`
   (`backend/reporting/run_signals.py`), so `emit_indexer_report`
(`backend/reporting/__init__.py`,
   already called in `finally` from `main.py` / `worker/__main__.py` / `orchestration.py`) folds
it
   into the report. Include the requested-vs-chosen encoder + reason (`probe_failed`) in the
payload.
   - _Wiring note:_ `VideoCompiler.compile_highlights(raw_video_path, segments,
output_filename)` does
     **not** currently receive `video_id`. Two clean options for the implementer:
     - (a) thread `video_id` (optional) into `compile_highlights` and emit from the fallback
site; or
     - (b) surface the chosen encoder back to the caller (return value / attribute) and let the

orchestration/worker layer ? which already holds `video_id` **and** the configured encoder ?
compare and emit. (b) keeps the compiler reporting-agnostic and is probably preferable.
2. **Forensic runlog event (optional, complementary)** ? also record `RunTelemetry.event("encoder_fallback", requested=..., chosen="libx264")` (`backend/utils/run_telemetry.py`) when a telemetry handle is active, so the black-box runlog shows it at the moment of the stitch.
3. **Monitoring / alert** ? expose the per-video report signal to whatever harvests reports (`backend/reporting/harvest.py` / obsreport) and add an alert that trips when `gpu_encoder_fallback_to_cpu > 0` on an NVENC-configured deployment. A cheap first cut: a WARN/ERROR structured log line with a stable, greppable event key (e.g. `event=gpu_encoder_fallback_to_cpu`) that an external log-based monitor can alert on, even before the full report-harvest path is wired.

## Acceptance criteria / DoD

- [ ] A hardware?CPU encoder fallback emits a structured, harvestable signal (not just a log line),
keyed to the run/video, carrying `requested`, `chosen`, and `reason`.
- [ ] No signal is emitted on a deployment legitimately configured for libx264 (no false positives).
- [ ] The signal is surfaced somewhere a monitor can alert on it (report harvest and/or a stable structured log event key), with the alert scoped to NVENC-configured deployments.
- [ ] Unit test: fallback path emits the signal exactly once with the right payload; the libx264 config path emits nothing. (Extend `tests/test_compiler.py`, alongside #527's probe tests.)
- [ ] A one-line note in the relevant observability doc so the alert is discoverable.

## Refs

- Caused-the-need: #521 (NVENC-on-L4), #525 (merge), **#527** (the silent-fallback bug that motivated this).
- Related observability / perf: #331 (hosted processing CPU-bound; GPU/NVENC levers), #326 (hosted-app observability: tracing + replay).
- Code: `backend/pipeline/stitcher/compiler.py::_select_encoder`, `backend/reporting/run_signals.py`, `backend/reporting/__init__.py::emit_indexer_report`, `backend/utils/run_telemetry.py::RunTelemetry.event`.

#### 128. user (2026-07-08T06:43:27.532Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer/b55cb444-5f5e-4770-ad71-cee76683ff71/scratchpad/issue_body.md (file state is current in your context ? no need to Read it back)

#### 129. assistant (2026-07-08T06:43:34.738Z)

Sibling session updated MEMORY.md (PR #528) ? noted, leaving it. Now let me file the issue with the `bug` label (per your "file a bug" framing).

#### 130. user (2026-07-08T06:43:40.289Z)

https://github.com/Khelsutra/badminton-highlight-indexer/issues/529

#### 131. assistant (2026-07-08T06:44:04.224Z)

Filed: **[issue #529](https://github.com/Khelsutra/badminton-highlight-indexer/issues/529)**. Let me tie it back to PR #527 with a short comment (helps whoever reviews the fix see the monitoring follow-up) and record it in memory.

### 132. user (2026-07-08T06:44:10.854Z)

https://github.com/Khelsutra/badminton-highlight-indexer/pull/527#issuecomment-4912085140

### 133. user (2026-07-08T06:44:13.939Z)

```
18      **Why:** empirically (owner's RTX 2070 Super, ffmpeg 8.1.1) h264_nvenc probe FAILs at
64x64 & 128x128
19      but PASSes at ~145px+ (256/320 all pass, 4K works). Likely never caught because the
2026-07-08 CUJ
20      that motivated #521 measured the libx264 control-plane path, not a live L4 NVENC stitch.
21
22      **How to apply:** verified locally green ? full suite (1878 passed / 8 skipped), `ruff
check .`,
23      `mypy backend training` (2.1.0). **Still owner-gated:** confirm on a real NVENC box that
h264_nvenc
24      now probes PASS at 256x256 AND that a live L4 stitch actually emits `-c:v h264_nvenc`
(not the
25      libx264 fallback) ? I have no GPU to confirm the live encode.
26
27      **Repo hygiene note (2026-07-08):** did this in an isolated `git worktree`
28      (`../bhi-nvenc-probe-wt`) because the main checkout had conflicted sibling WIP ? `UU`
marks on
29      `orchestration.py` + `compiler.py`, a `sd-reel-engine-wip` stash, branch
`codex/issue-523-sd-reel-plan`,
30      **no merge actually in progress**. That WIP was left untouched.
31
```

### 134. user (2026-07-08T06:44:20.977Z)

```
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-
highlight-indexer\memory\pr-527-nvenc-probe-resolution.md has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

### 135. assistant (2026-07-08T06:44:39.027Z)

Done. Filed **[issue #529 ? [observability] Alert/metric when the GPU (NVENC) stitch silently falls back to CPU (libx264)](https://github.com/Khelsutra/badminton-highlight-indexer/issues/529)** (`bug` label), and cross-linked it from PR #527.

I grounded it in the repo's **actual** observability conventions rather than inventing a mechanism:

- **The signal** ? emit `run_signals.incr(video_id, "gpu_encoder_fallback_to_cpu")` on fallback, mirroring how `gemini.py` already reports `oversize_clips_skipped` / `generate_failed_chunks`. That folds it into the per-video report via `emit_indexer_report` (already called in `finally` across `main.py` / `worker` / `orchestration`), which is the harvestable/monitorable surface ? plus an optional `RunTelemetry.event(...)` for the forensic runlog.
- **The alert scoping** (the part that keeps it low-noise) ? fire **only** when the deployment is *configured* for NVENC but the run chose libx264. A box legitimately set to `encoder=libx264` never probes and must produce no signal. So the alert means exactly *"a machine that should have hardware encode is silently on CPU"* ? misconfig, broken driver/ffmpeg, or a #527-class regression.
- **Wiring note for the implementer** ? `compile_highlights` doesn't currently receive `video_id`, so I flagged the two clean options (thread `video_id` in, or surface the chosen encoder back to the caller that already holds `video_id` + the configured encoder ? the latter keeps the compiler reporting-agnostic).
- **DoD** includes a unit test that the fallback emits exactly once and the libx264-config path emits nothing (no false positives), and a doc note so the alert is discoverable.

Refs wired up: caused-the-need #521/#525/**#527**; related observability/perf #331 (CPU-bound levers) and #326 (hosted-app observability). No duplicate existed. Recorded the follow-up in memory alongside the #527 note.

This is exactly the guardrail that would have caught the original bug on its first L4 run instead of letting it hide for weeks.

**136. user (2026-07-08T06:46:48.478Z)**

merged

**137. assistant (2026-07-08T06:47:08.481Z)**

Congrats ? that closes out the fix. Let me verify the merge landed cleanly on `master` and then update the tracking state.

**138. user (2026-07-08T06:47:26.689Z)**

```
 - [deleted]         (none)     -> origin/fix/hosted-local-detection-guardrail
 - [deleted]         (none)     -> origin/fix/nvenc-probe-resolution
   dac6736..65ee4c6  master     -> origin/master
=== PR #527 state ===
state=MERGED  base=master  mergedAt=2026-07-08T06:46:27Z  mergeCommit=65ee4c64dcab
=== is the fix on origin/master now? ===
65ee4c6 fix(serving): probe NVENC at 256x256, not 64x64, so it isn't falsely rejected
(#521/#525) (#527)
=== remote branch still there? ===
```

**139. assistant (2026-07-08T06:47:45.171Z)**

Merge confirmed: PR #527 is **MERGED** into `master` as squash commit [`65ee4c6`](https://github.com/Khelsutra/badminton-highlight-indexer/commit/65ee4c64dcab), `origin/master` advanced `dac6736 ? 65ee4c6`, and the remote branch auto-deleted. Let me clean up my local branch ref and refresh the tracking memory (re-reading MEMORY.md first since sibling sessions edit it concurrently).

**140. user (2026-07-08T06:47:47.577Z)**

```
=== local branch ref present? ===
7d6c049f270116fecd1cf4b6976150a02d57b1e9
Deleted branch fix/nvenc-probe-resolution (was 7d6c049).
```

**141. user (2026-07-08T06:47:47.596Z)**

```
1       # Memory index
2
3       - [PR #525 ? compile-on-L4 (#521)](pr-525-compile-on-l4.md) ? config-driven
phase?machine placement + L4 NVENC stitch; Part 3 (cheaper watermark) deferred.
4       - [PR #527 ? NVENC probe 64x64?256x256](pr-527-nvenc-probe-resolution.md) ? sub-minimum
probe silently defeated #521's NVENC-on-L4 win; open, awaits owner live-GPU confirm.
5       - [PR #528 ? worker mirror-required](pr-528-worker-mirror-required.md) ? #521 follow-up:
L4-worker stitch now fails on a lost reel mirror-upload (was silent 404-on-success); found by
adversarial review, CI green.
6
```

**142. assistant (2026-07-08T06:47:55.725Z)**

Local branch deleted (was merged as squash `65ee4c6`). Now updating the tracking memory to reflect merged state ? while preserving the honest caveat that merging ? live-GPU verification.

**143. user (2026-07-08T06:47:58.609Z)**

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\memory\MEMORY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

**144. user (2026-07-08T06:48:07.163Z)**

<tool_use_error>String to replace not found in file.
String: description: PR #527 fixes the NVENC availability probe (64x64?256x256) that silently defeated #521's NVENC-on-L4 win; open, awaits owner live-GPU confirmation.
(note: Edit also tried swapping \uXXXX escapes and their characters; neither form matched, so the mismatch is likely elsewhere in old_string. Re-read the file and copy the exact surrounding text.)</tool_use_error>

### 145. assistant (2026-07-08T06:48:10.288Z)

Let me re-read the file's frontmatter to get the exact text.

### 146. user (2026-07-08T06:48:11.306Z)

```
1       ---
2       name: pr-527-nvenc-probe-resolution
3       description: PR
4       metadata:
5         node_type: memory
6         type: project
7         originSessionId: b55cb444-5f5e-4770-ad71-cee76683ff71
8       ---
9
10      [PR #527](https://github.com/Khelsutra/badminton-highlight-indexer/pull/527) (branch
11      `fix/nvenc-probe-resolution`, off `origin/master`) fixes a bug in
`_probe_encoder_available`
12      (`backend/pipeline/stitcher/compiler.py`, added by #521 / merged via #525, see
[[pr-525-compile-on-l4]]):
13      the 1-frame NVENC probe encoded from `color=c=black:s=64x64:d=0.1`, but **64x64 is below
NVENC's
14      minimum encode dimensions**, so on a real GPU the probe FAILS even when NVENC works at
real
15      resolutions ? `_select_encoder` silently falls back to libx264 ? #521's NVENC-on-L4 4K
re-encode
16      win is defeated. Fix: bump to a `_PROBE_SOURCE_SIZE = "256x256"` constant.
17
```

### 147. assistant (2026-07-08T06:48:18.759Z)

The memory subsystem normalized the frontmatter (collapsed `description` to "PR"). Let me restore a meaningful description reflecting the merged state.

### 148. user (2026-07-08T06:48:20.914Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\memory\pr-527-nvenc-probe-resolution.md has been updated successfully. (file state is current in your context ? no need to Read it back)

### 149. user (2026-07-08T06:48:31.874Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\memory\pr-527-nvenc-probe-resolution.md has been updated successfully. (file state is current in your context ? no need to Read it back)

### 150. assistant (2026-07-08T06:48:52.807Z)

Verified and cleaned up. PR #527 is merged as squash commit `65ee4c6` on `master`; I deleted the stale local branch (remote auto-deleted) and updated the tracking memory.

Two threads stay open ? and I want to be precise that **merged ? live in production**:

1. **Live-GPU verification is still outstanding.** The code + tests are correct and on master, but nothing has yet exercised a real NVENC encode. Per the release-baseline note now in memory, there's an extra gate: the deployed worker image (`:latest`) predates the compile-on-L4 subcommand, so this only becomes live after a **worker image rebuilt from this master** ? then

confirm on a real box that `h264_nvenc` probes PASS at 256×256 and a live L4 stitch actually emits `-c:v h264_nvenc`.

2. **[Issue #529](https://github.com/Khelsutra/badminton-highlight-indexer/issues/529) (monitoring) is still open** ? that's the guardrail you asked for: an alert on every silent GPU?CPU fallback. It's worth landing *before* the rebuilt image ships, so the first live L4 run either confirms NVENC is engaged or trips the alert ? instead of another silent stretch on CPU.

Net: the bug is fixed and merged; the fix's real-world payoff is confirmed only once the worker image is rebuilt and one live stitch is observed. Nice work getting it in.